Security and Robustness of Neural Code Systems: A Survey of Attack Surfaces, Defenses, and Evaluation

BIN DUAN, The University of Queensland, Australia
GUOWEI YANG, The University of Queensland, Australia

Neural code systems face attacks through program inputs, training data, and the external context consumed by coding agents. We review 148 works available by 30 June 2026, comprising 219 evaluated contributions across attacks, risk measurements, defenses, and evaluation resources. We organize attacks by their entry into training state, current inference input, or runtime context, and examine output, privacy, and ecosystem risks separately. Our analysis connects each reported result to its outcome oracle and, for program transformations, its preservation contract. Construction arguments, paired tests, and local symbolic checks support different retained properties. Execution may assess benign utility, a malicious payload, or an agent action, with different implications for security. Within-study comparisons show that protection depends on the attack family and can reduce functional utility. The synthesis identifies opportunities for executable preservation collections, shared agent traces, adaptive evaluation, and versioned benchmarks.

CCS Concepts: • **Security and privacy** → **Software and application security**; • **Software and its engineering** → *Software verification and validation*; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: Neural Code Models, Code LLMs, Coding Agents, Adversarial Attacks, Backdoor Attacks, Data Poisoning, Prompt Injection, Runtime Context, Robustness, Secure Code Generation

1 Introduction

Neural code systems are used to classify programs, retrieve code, generate implementations, and carry out development tasks through external tools. Security studies have expanded with these capabilities. Early work examined whether changes that retain a program's behavior could alter a code model's prediction [181, 184]. Studies of code LLMs subsequently investigated insecure suggestions, reproduction of training material, and poisoned completions [112, 129, 196]. Coding agents add repositories, retrieval stores, tool descriptions, and execution environments to this setting, allowing attacker-controlled content to influence actions as well as generated text [91, 166].

The experiments differ in both attacker control and measured outcome: a changed prediction, an insecure suggestion, and an executed command are different security events. Existing surveys organize this literature by problem class, system type, security property, or lifecycle stage (Table 1). We connect these perspectives through the state an adversary can modify and the property an evaluation measures.

Our corpus contains 148 works available by 30 June 2026 and 219 evaluated contributions: 75 attacks, 30 risk studies, 110 defenses, and 4 evaluation resources. A work can contribute several roles, such as an attack and its evaluated mitigation. The survey contributes:

- A system model relating task-specific models, code LLMs, and coding agents to three attacker entry points: training state, current inference input, and runtime context (RQ1).

Authors' Contact Information: Bin Duan, b.duan@uq.edu.au, The University of Queensland, Brisbane, Queensland, Australia; Guowei Yang, guowei.yang@uq.edu.au, The University of Queensland, Brisbane, Queensland, Australia.

Table 1. Related surveys by system coverage and organizing perspective. Checkmarks indicate substantive coverage; "–" marks system forms outside the primary scope.

Survey	Task-specific code model	Code LLM	Coding agent	Organizing perspective
Qu et al. [120]	✓	–	–	Attacker knowledge, attack scenario, victim task, perturbation type, and evaluation metric.
Y. Yang et al. [172]	✓	✓	–	Security property, attack family, threat model, and countermeasure.
Chen et al. [25]	✓	✓	–	Models, attacks, defenses, datasets, metrics, and artifacts.
Z. Yang et al. [175]	–	✓	–	Non-functional properties, including robustness, security, privacy, explainability, efficiency, and usability.
Negri-Ribalta et al. [104]	–	✓	–	Security risks introduced by AI-generated code and possible mitigations.
Basic and Giaretta [6]	–	✓	–	Insecure code generation, vulnerability detection and repair, prompting strategies, and data poisoning.
Qu et al. [119]	–	✓	–	Training settings, trigger designs, backdoor attacks, and defenses.
She et al. [131]	✓	✓	–	Pitfalls in data, system design, evaluation, and deployment.
Maloyan and Namiot [95]	–	–	✓	Prompt injection through skills, tools, and protocol ecosystems.
This survey	✓	✓	✓	Classification of attacks by entry through the training state, current inference input, or runtime context; separate treatment of studies that observe existing risks; reported outcome measures and program-preservation checks.

- A comparison of attack mechanisms at these entry points and measurements of output, privacy, and ecosystem risks (RQ2).
- An analysis of defenses by protection target, mechanism, utility cost, and evaluated outcome (RQ3).
- A synthesis of preservation checks, outcome oracles, benchmarks, and the conditions for comparing results (RQ4).

Section 2 describes the review approach. Sections 3–6 answer the RQs, followed by threats to validity, research opportunities, and conclusions.

2 Approach

The review combines database searches, supplementary source checks, and snowballing to answer four questions.

2.1 RQs

RQ1: Systems and boundaries. Which system forms are evaluated, and where can attacker-controlled material enter?

RQ2: Attacks and risks. What mechanisms induce failures, and what existing security risks are measured?

RQ3: Defenses. What protection mechanisms are evaluated, against which threats and at what utility cost?

RQ4: Evaluation. What preservation checks, outcome oracles, resources, and protocols support the reported results?

2.2 Search Strategy

The initial collection used DBLP, arXiv, and conference and journal indices. Its exports and candidate-level screening log were not retained. A supplementary search on 19 October 2026 used a public web index restricted to DBLP and arXiv, retaining the availability cutoff of 30 June 2026. The companion package contains its 16 exact queries, returned entries, screening decisions, and source follow-ups.

Searches combined system terms (*code model*, *code LLM*, *neural code*, *source code*, *code generation*, and *coding agent*) with adversarial robustness, poisoning, backdoors, prompt injection, privacy, and secure-generation terms. Supplementary queries added coding assistants, runtime enforcement, retrieval, tool and skill injection, provenance, unlearning, and package hallucination. Strings were adapted to each source's search syntax.

The supplementary index returned 107 distinct entries: 18 author, venue, disambiguation, or documentation pages; 21 studies already in the corpus; 17 newly included works; five secondary reviews; five post-cutoff works; 40 scope exclusions; and one code-related work with unresolved pre-cutoff availability. Eligibility and protocol coding followed primary sources linked from these returns.

2.3 Study Selection

2.3.1 Eligibility Criteria. Primary works were eligible when they evaluated a task-specific code model, code LLM, or coding agent on a code-related attack, empirical security risk, defense, or repository-level resource reporting functional and security outcomes. Eligible sources included archival publications, preprints, and first-party technical reports with inspectable methods and measured results. We excluded generic LLM or agent studies without separate code-related results, conventional security techniques without a neural component, unevaluated proposals, and superseded versions. Secondary surveys informed discovery and positioning.

2.3.2 Screening and Version Selection. The initial collection used title/abstract screening and full-text assessment. Supplementary screening and primary-source checks were model-assisted. The records distinguish initial coding from subsequent changes and retain unresolved eligibility decisions. Final verification excluded five works, comprising seven contributions, whose complete protocols or correspondence to a pre-cutoff version could not be verified; the exclusion ledger gives the reasons.

AUTHOR TODO 01 / 作者待办：补充实际使用的工具与任务

请写明检索、文献筛选、贡献编码和来源核查分别使用的 AI 工具/模型、版本或访问日期，以及输入材料、判断规则和输出如何采用。按实际日志补充；已有任务规则见本节和 codebook.md，工具记录见 protocol_audit.csv 的 protocol_reviewer 字段。请确认这些记录是否覆盖了全部实际使用。

Deduplication and version handling. We matched titles, authors, DOIs, and arXiv identifiers, merging preprints and archival publications into one work. Evidence uses versions available by 30 June 2026. ACW uses its August 2025 arXiv version 4, SET-Backdoor its

December 2025 preprint, and scanner-feedback repair its June 2025 version; later scientific additions are excluded.

2.4 Snowballing Search

Backward and forward snowballing supplemented the initial collection; separate hit counts were not retained. Logged reference and primary-source follow-ups added Quiring et al.'s authorship attack, FunPoison, and Signature [77, 121, 165]. Their discovery and eligibility records are separate from the 107 index returns.

2.5 Data Extraction and Analysis

The corpus contains 148 works and 219 evaluated contributions. Each contribution represents a work and study role; attacks are split further when a work evaluates different entry surfaces. Variants, victim configurations, and defense baselines within one work-role evaluation remain grouped. FDI, for example, contributes retrieval and continual-training attacks plus a grouped screening evaluation. Un-Repair's proposed evaluation and Tambon et al.'s general functional-bug taxonomy remain background references [61, 146].

We assign four roles: *attack* for an intervention intended to induce failure, including fixed malicious-request benchmarks; *risk study* for measurements of insecure generation, training-data exposure, or ecosystem properties; *defense* for evaluated prevention, detection, or mitigation; and *evaluation resource* for repository- or project-level benchmarks reporting functionality and security. Diagnostic membership and extraction probes belong to the risk role because they measure existing training-data relationships. The resulting records comprise 75 attacks, 30 risk studies, 110 defenses, and 4 resources.

Each record contains the study role, system form, mechanism, task, representation, result type, outcome oracle, and applicable attacker access and preservation checks (Table 2). Result codes are PRED for labels, rankings, similarity scores, or detector decisions; GEN for generated code or text; MIX for both; AGT for evaluated actions; and OBS for artifact measurements without a victim-model query. Result type describes the measured output: SecRepoBench evaluates agent-returned code (GEN), while RedCode also evaluates agent actions (AGT). Sections 6.1.1 and 6.1.2 define preservation and outcome codes.

Model knowledge is WB (white-box), GB (gray-box), or BB (black-box). Write access is TDW for training data, MW for model weights, or RCW for runtime context. Current-request control is represented by the inference-input surface.

The first author coded the initial table in two passes. Model-assisted primary-source checks subsequently covered all 219 records from 148 works: 205 protocol checks and 14 targeted checks (Table 14). The companion data retain source versions, locators, decisions, and an author-review sheet; human confirmation remains pending. Author manuscripts and official experiment artifacts supply evidence attributed to their named versions. Counts and table views are generated from the active records.

Table 2. Extracted information and the RQs it supports.

RQ	Information used in the synthesis	Analysis
1	System form, result type, writable state, and entry point	System and boundary map
2	Attack or risk role, mechanism, task, representation, access, and budget	Mechanism comparison
3	Defense role, protection target, control mechanism, and reported utility	Defense coverage
4	Benchmarks, preservation contract and support, outcome oracle, execution purpose, and source version	Evaluation comparison

AUTHOR TODO 02 / 作者待办：补充真实的人工复核过程

请核实前文"第一作者两轮编码"是否符合实际，并补充：谁复核、核对哪些记录或字段、全量还是抽样及其数量/分母、依据哪些原文、如何处理分歧。对已完成的核验，在 author_review.csv 填写 author_decision、author_correction、author_initials 和 review_date。未完成的记录继续留空；完成后据实更新本段的待确认状态。

3 RQ1: What Neural Code Systems and Security Boundaries Have Been Studied?

The reviewed systems range from program prediction to generation and iterative tool use. Each exposes different state and interfaces to an adversary.

3.1 Which System Forms Are Evaluated?

Task-specific models provide services such as classification, retrieval, and summarization. Code LLMs support code and text tasks through prompting. Coding agents add repository state, retrieval, tools, and an execution loop (Table 3). These are evaluated configurations: the same model can serve a task-specific interface or operate within an agent. Generation introduces sampled executable outputs; agents additionally turn outputs into actions on external state.

3.2 Where Does Attacker-Controlled Material Enter the System?

We classify attacks by the first point at which attacker-controlled content or state enters the evaluated victim path (Figure 1):

Training state. Data used in pre-training, fine-tuning, instruction tuning, or continual updates; modified weights; or a distributed model.

Current inference input. Code, prompts, comments, instructions, or accessible continuous prefixes supplied for the current request. An optimized prefix applied to a fixed model enters here.

Runtime context. Repository files, retrieval corpora, dependencies, skills, tool descriptions, or outputs consumed outside the direct request.

A malicious comment can enter through any of these surfaces, depending on whether it is trained on, submitted directly, or planted for later consumption. FDI evaluates both

Table 3. Neural code system forms and the security boundary introduced by each form.

System form	Typical interface	State beyond the current request	Security consequence
Task-specific code model	Label, ranking, similarity, or summary	Model parameters and training data	Robustness depends on the permitted code transformation, model access, and task metric.
Code LLM	Completion, patch, explanation, or generated program	Model parameters, training data, and sometimes retrieved context	Security includes generated-code functionality, weakness detection, memorization, and sampling policy.
Coding agent	Iterative observations, plans, tool calls, and actions	Repository state, retrieval stores, rules, skills, tool descriptions, credentials, and execution environment	Security depends on the full trust path from external context to authorization, execution, and terminal effect.

retrieved and training feedback and therefore receives two attack records. Risk studies have a separate role and retain their diagnostic query mechanisms.

Classification follows the evaluated entry point. PELICAN modifies an inference-time binary to activate naturally learned triggers [190]; CodeJailbreaker submits a simulated commit message in the current prompt [111]. Both are input attacks.

AUTHOR TODO 03 / 作者待办：核验分类边界与统计口径

请对照主 CSV 的来源定位确认：PELICAN、CodeJailbreaker 属当前输入攻击；FDI 的训练更新与检索路径分为两项攻击；隐私诊断归风险研究。核对攻击分组 46/22/7 与 148 篇、219 项贡献的统计口径，确认多角色记录不是重复计数，并将实际核验决定填回 author_review.csv。

3.3 How Does Research Coverage Differ across Boundaries?

Table 4 aligns attack and risk records with defense protection targets. Runtime context has seven attack records and four defense evaluations, each using particular retrieval or agent settings. Input robustness and training compromise have larger literatures, although their evaluation protocols also vary.

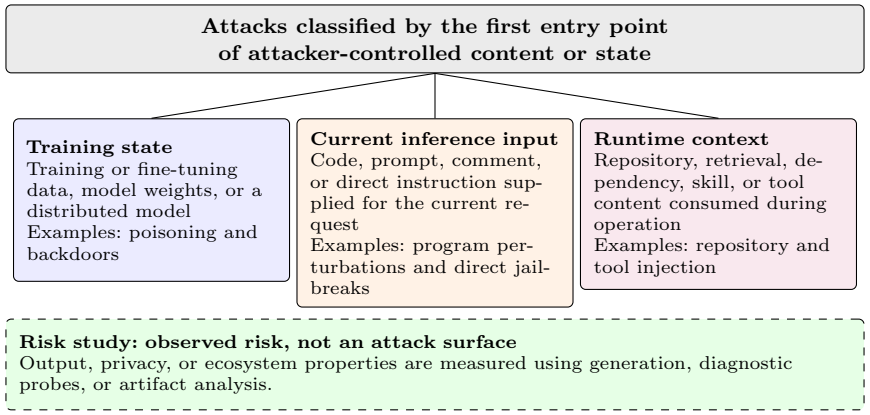


Fig. 1. Classification of attacks by the first point at which attacker-controlled content or state enters the victim system. Risk studies form a separate group for measurements of output, privacy, and ecosystem properties, including diagnostic privacy probes.

Table 4. Contribution counts by attack surface or risk and primary defense target.

Attack surface or risk	Attack/risk records	Defense records	Interpretation
Current inference input	46 attacks	38	Transformation defenses span prediction and generation with task-specific oracles.
Training state	22 attacks	29	Screening, denoising, and unlearning vary in triggers, budgets, and persistence tests.
Runtime context	7 attacks	4	Context screening, guards, and skill scanners cover particular injection settings.
Output and ecosystem	30 risk studies	39	Secure generation, privacy, provenance, and unlearning protect different properties.

4 RQ2: What Attacks and Security Risks Have Been Evaluated?

We compare attacks on current inputs, training state, and runtime context by their modified material, construction mechanism, access, and target behavior, then examine output, privacy, and ecosystem risks.

4.1 How Are Current Inference Inputs Manipulated?

Tables 5 and 6 compare adversarial construction for programs and representations, then prompts and comments. Table 7 covers fixed transformation suites and malicious-request collections.

4.1.1 Adversarial Attacks Using Program, Binary, and Feature Transformations. These attacks modify source programs, binaries, graph representations, or features. The perturbation type specifies what can change; the selection procedure chooses the location and replacement.

Table 5. Program, binary, and feature attacks: modified material, candidate selection, and evaluated access. Multiple labels indicate mixed perturbations or access settings.

Study	Venue	Target task	Perturbation type	Selection procedure	Attacker access
Quiring et al. [121]	USENIX Sec.'19	Authorship attribution	Identifier, style, control-flow and API transformations	Monte Carlo tree search; rejects variants failing shared-input output checks.	Black-box
Yefet et al. [181]	OOPSLA '20	Method-name prediction; variable-use prediction	Identifier replacement; unreachable-code insertion	Uses victim-model gradients to choose transformations.	White-box
Zhang et al. [184]	AAAI'20	Code functionality classification	Identifier replacement	Uses Metropolis–Hastings search guided by victim-model outputs.	Black-box
Bielik and Vechev [9]	ICML'20	Static type inference	Identifier replacement; literal replacement; structural transformation	Searches semantics-preserving edits against the victim model.	White-box
Springer et al. [137]	arXiv'20	Method-name prediction	Identifier-subtoken replacement	Ranks replacement tokens by embedding norms or training-corpus frequency.	White-box and black-box
Srikant et al. [138]	ICLR'21	Method-name prediction	Structural transformation	Jointly optimizes transformation locations and choices.	White-box
Vahdat Pour et al. [151]	ICST'21	Method-name prediction; code captioning; code search; documentation generation	Structural transformation	Uses mutation-guided search over behavior-preserving refactorings.	Gray-box
Henkel et al. [50]	SANER'22	Method-name prediction	Structural transformation	Uses gradients to select parameters of semantics-preserving transformations.	White-box
Yang et al. [174]	ICSE'22	Vulnerability detection; clone detection; authorship attribution	Identifier replacement	Uses greedy search followed by a genetic algorithm.	Black-box
Zhang et al. [183]	TOSEM'22	Functionality classification; clone detection; defect prediction	Identifier replacement; structural transformation	Uses victim-model feedback to guide hill climbing.	White-box and black-box
Zhou et al. [195]	TOSEM'22	Code-comment generation	Identifier replacement	Ranks identifiers and substitutes semantically similar alternatives.	Black-box
Li et al. [86]	ICSE'22	Authorship attribution	Coding-style transformation	Selects behavior-preserving style rewrites that change the prediction.	Black-box
Jia et al. [68]	arXiv'22	Binary-code similarity detection	Binary-instruction insertion	Searches instruction insertions while preserving the control-flow graph.	Black-box
Zhang et al. [190]	USENIX Sec.'23	Binary analysis	Trigger-instruction insertion	Inverts triggers in naturally trained models and synthesizes blocks checked for symbolic equivalence.	White-box and black-box
Wei et al. [162]	IEEE Trans. Reliab.'23	Code summarization; method-name prediction	Source-code mutation	Uses coverage-guided fuzzing over ten mutation operators.	White-box
Jha and Reddy [66]	AAAI'23	Code translation; program repair; code summarization	Code-token replacement	Ranks influential tokens and obtains replacements from a pretrained code model.	Black-box

continued on next page

Table 5 (continued)

Study	Venue	Target task	Perturbation type	Selection procedure	Attacker access
Tian et al. [147]	ASE'23	Vulnerability detection; clone detection; authorship attribution; functionality classification; defect prediction; defect-type prediction	Identifier replacement; structural transformation	Uses differences from a reference program to select modifications.	Black-box
Na et al. [102]	ACL'23	Clone detection; vulnerability detection; authorship attribution	Unreachable-code insertion	Ranks locations and inserts a candidate statement through an unused variable.	Black-box
Zhang et al. [187]	EMNLP Findings'23	Clone detection; vulnerability detection; authorship attribution; functionality classification	Identifier replacement	Searches nearby candidates in a continuous representation space.	Black-box
Chen et al. [22]	EMNLP Findings'23	Code summarization; code translation; clone detection; defect prediction	Structural transformation	Uses the abstract syntax tree to guide transformations.	Black-box
Nguyen et al. [107]	arXiv'23	Authorship attribution; vulnerability detection; clone detection	Graph-pattern insertion; unreachable-code insertion	Probes the victim, instantiates discriminative syntax-tree patterns, and inserts them as code.	Black-box
Du et al. [33]	FSE'23	Vulnerability detection; clone detection; code summarization	Identifier replacement	Prioritizes identifiers by statement context and searches replacement sequences with beam search.	Black-box
Zhang et al. [182]	ASE'24	Method-name prediction	Identifier replacement; structural transformation; unreachable-code insertion	Constructs variants against a white-box source model and transfers them to the target.	White-box source; black-box target
Capozzi et al. [12]	IEEE Access'24	Binary-code similarity detection	Binary-instruction insertion	Uses spatially guided greedy search or gradient-guided selection.	White-box and black-box
Zhou et al. [193]	FSE'24	Defect prediction; clone detection; authorship attribution; code translation; code summarization	Identifier replacement	Uses multi-objective evolutionary search over context-aware candidates.	Black-box
Zhang et al. [185]	JSEP'24	Functionality classification; clone detection	Identifier replacement	Masks tokens and obtains context-compatible replacements from a pretrained code model.	Black-box
Chen et al. [23]	J. Softw.'24	Vulnerability detection	Type, name, operator, control-flow and literal rewrites; statement insertion	Uses PPO to select among nine AST-located transformation operators.	Black-box
Liu et al. [89]	USENIX Sec.'24	Vulnerability detection	Code-snippet insertion	Uses attention, language-model-generated snippets, and fuzzy genetic search.	Black-box
Yang et al. [171]	IST'25	Code translation	Syntactic and structural transformation	Applies a predefined transformation suite before translation.	Black-box
Huang et al. [54]	ICSE'25	Vulnerability detection; clone detection; code summarization	Identifier replacement	Uses Bayesian optimization informed by previous victim-model queries.	Black-box

continued on next page

Table 5 (continued)

Study	Venue	Target task	Perturbation type	Selection procedure	Attacker access
Awal et al. [5]	SQJ'25	Just-in-time defect prediction; clone detection; code-review usefulness classification	Input-feature modification	Changes the highest-ranked features identified by model explanations.	Black-box
Chen et al. [21]	arXiv'25	Binary-code similarity detection	Binary-instruction perturbation	Uses model-agnostic explanations to locate instructions for modification.	Gray-box
Rath et al. [123]	TPS-ISA'25	Vulnerability detection	Graph-oriented structural transformation	Applies natural, semantics-preserving transformations to live code.	Black-box (labels)

Identifier attacks share a perturbation unit but differ in search: Metropolis–Hastings (MHM), greedy and genetic search (ALERT), representation search (RNNS), statement-aware beam search, and Bayesian optimization select different candidates [33, 54, 174, 184, 187]. ACCENT uses identifier importance and similarity, CodeAttack modifies code tokens, and CodeBERT-Attack uses pretrained suggestions [66, 185, 195].

Structural attacks modify statements, control flow, style, or inserted code. Their selection mechanisms include gradients, parameter search, mutations, reference-code differences, reinforcement learning, and transfer [23, 50, 138, 147, 151, 182, 183]. DAMP and DIP insert unreachable code, GraphCodeAttack instantiates graph patterns, and EaTVul inserts generated snippets [89, 102, 107, 181].

Preservation depends on the complete operator set and task. STAB removes AST subtrees; RLAttack includes type widening and print insertion [22, 23]. Valid renaming rules cannot justify all these operators. Bielik and Vechev instead preserve a type-inference label under a stated transformation relation [9].

Binary attacks perturb instructions for similarity detection [12, 21, 68]; feature attacks edit explanation-selected model inputs [5]. NatGVD modifies source code to alter its graph representation [123]. These representations require different validity checks.

White-box selection can use gradients and internals; black-box selection uses labels, scores, query history, or transfer. STRATA, CARROT, and SpatialGreedy evaluate multiple access settings [12, 137, 183]. Transfer experiments distinguish an accessible source model from a black-box target [182].

4.1.2 Adversarial Attacks Using Prompts and Comments. These attacks modify text, code context, comments, simulated commit messages, or continuous prefixes supplied with a generation request (Table 6).

Table 6. Adversarial attacks using prompts, comments, and continuous prefixes.

Study	Venue	Target task	Modified material	Selection procedure	Attacker access
He and Vechev [47]	CCS'23	Vulnerability-inducing generation	Continuous prefix	Optimizes vulnerability-steering prefix vectors while keeping victim weights fixed.	White-box
Wu et al. [164]	arXiv'23	Vulnerability-inducing code generation	Natural-language prefix and suffix	Uses evolutionary search to construct benign-looking attack text.	Gray-box
Li et al. [83]	ASE'24	Vulnerability-inducing code completion	Source-code completion prompt	Uses gradient attribution to select locations, then applies code mutations.	White-box

Hossen and Hei [53]	arXiv'24	Function synthesis	Natural-language suffix	Generates and appends suffixes that reduce output quality.	Black-box
Jenko et al. [65]	ICML'25	Vulnerability-inducing code completion	Code comment	Optimizes a short attack string through repeated victim-model queries.	Black-box
Ouyang et al. [111]	ICSE'26	Malicious-code generation and completion	Simulated commit message in the current prompt	Constructs a software-evolution request with malicious intent encoded in the commit-message field.	Black-box

DeceptPrompt searches for vulnerability-inducing prefixes and suffixes, while Degrade-Prompter reduces functional quality through suffixes [53, 164]. AdvPro mutates completion context, INSEC optimizes a comment string, and CodeJailbreaker encodes malicious intent in a directly submitted commit-message prompt [65, 83, 111]. SVEN optimizes continuous prefixes through internal access to a fixed victim, requiring a stronger interface than a text-only API [47].

4.1.3 Inference-Input Robustness and Safety Benchmarks. These studies supply predefined transformations or request collections, rather than searching separately for each victim input (Table 7).

Table 7. Inference-input robustness and safety benchmarks. The first three apply transformed inputs; the remaining rows include malicious or vulnerability-inducing requests.

Study	Venue	Benchmark type	Target task	Input construction
Wang et al. [158]	ACL'23	Transformed-input robustness	Function synthesis and completion	Applies fixed transformations to documentation, identifiers, syntax, and formatting.
Lam et al. [76]	NeurIPS '25	Transformed-input robustness	Program-output prediction; input generation	Combines structural code transformations with misleading comments.
Gao et al. [36]	arXiv'25	Transformed-input robustness	Program-output prediction	Applies structural transformations and misleading comments, print statements, and reasoning hints.
Chen et al. [19]	ASE'24	Malicious-request safety	Malicious-code generation, translation, and completion	Supplies curated malicious requests in text-to-code and code-to-code forms.
Guo et al. [42]	NeurIPS '24	Malicious-request safety	Unsafe code execution; harmful-code generation	Supplies risky natural-language and code requests.
Li et al. [81]	ACL'25	Malicious-request safety	Malware-generation refusal	Supplies direct and jailbreak-modified malware-generation requests.
Agarwal et al. [1]	arXiv'24	Security-request subset	Vulnerability-inducing generation	Prompts for vulnerable examples within a broader code-hallucination collection; 243 security-labelled items.

Robustness Benchmarks Using Transformed Inputs. ReCode varies documentation, names, syntax, and formatting in generation prompts [158]. CodeCrash combines transformations and misleading comments for execution reasoning, while TReaT adds misleading print statements and reasoning hints for output prediction [36, 76].

Malicious-Request Safety Benchmarks. RMCBench tests malicious generation, translation, and completion; RedCode tests harmful generation and unsafe execution; MalwareBench includes direct and jailbreak-modified malware requests [19, 42, 81]. These requests need no benign source transformation. CodeMirage contributes 243 constructed security-labelled requests and a hallucination detector [1]; it is distinct from the later code-origin benchmark of the same name [43].

Findings for current-input attacks. Of 46 input-attack records, 39 construct adversarial inputs and 7 provide fixed collections. More elaborate search has not produced a uniform progression in preservation evidence: Quiring et al. already used shared-input execution checks in 2019 [121]. Feedback also varies within black-box settings: NatGVD uses labels, CodeBERT-Attack and ITGen use classification confidence, and generation attacks can compare sequences with references [54, 123, 185]. Prediction evasion, functional degradation, vulnerability induction, and unsafe compliance require separate success definitions.

4.2 How Is Training State Compromised?

Training compromise modifies data or weights before the attacked request, including through continual updates. *Poisoning* degrades or redirects behavior; a *backdoor* induces trigger-conditioned behavior while retaining ordinary utility (Table 8).

Table 8. Training-compromise attacks. *Acc*: TDW for training-data write and MW for model/weight modification. *PP*, *O*, and *I* use the codes defined in Sections 2, 6.1.1, and 6.1.2.

Method	Venue'Yr	Target	Mechanism	Acc	PP	O	I
You Autocomplete Me [129]	USENIX Sec.'21	Completion	Crafted files or altered weights induce insecure suggestions	TDW/ MW	–	R	GEN
TrojanPuzzle [2]	S&P'24	Code suggestion	Templated placeholder trigger; payload never verbatim	TDW	A	R+S+ X	GEN
CodeBreaker [170]	USENIX Sec.'24	Completion	LLM-obfuscated payload; evades SA + LLM detectors	TDW	A	R+L+ S+X	GEN
Cotroneo et al. [27]	ICPC'24	NL-to-code	Injected unsafe NL–code pairs (CWE patterns)	TDW	A	R+L	GEN
Generalized Adv. [125]	AsiaCCS'25	Completion	Group-trigger + learned embedding map	TDW	A	R+S+ X	GEN
You See What I Want [155]	FSE'22	Code search	Trigger tokens boost attacker-code ranking	TDW	A	R	PRED
BADCODE [140]	ACL'23	Code search	Target trigger in rare variable names	TDW	A	R	PRED
BadCS [116]	arXiv'23	Code search	Poisoned samples + weighted-loss KD	TDW	A	R	PRED
CodePoisoner [82]	TOSEM'24	Defect, clone, repair	Four trigger strategies; tree-sitter syntax screening	TDW	–	R+L+ C	MIX
AFRAIDOOR [176]	TSE'24	Summarization, method-name	Adaptive adversarial identifier rename	TDW	A	R+L	GEN
Ramakrishnan and Albarghouthi [122]	ICPR'22	Summarization	Fixed / grammatical dead-code triggers	TDW	A	R	GEN
Multi-target [85]	ACL'23	Multi-task (pre-train)	Task-agnostic pre-training triggers	TDW	A	R	MIX
Backdoor summarizers [156]	ASE'25	Summarization	Trigger + overwritten docstring; 20/300,000 in v1	TDW	A	R	GEN
SET-Backdoor [179]	arXiv'25	5 tasks / 6 languages	Behavior-preserving transformation triggers	TDW	A	R+L	MIX

continued on next page

Table 8 (continued)

Method	Venue'Yr	Target	Mechanism	Acc	PP	O	I
ToxicSQL [88]	SIGMOD'26	Text-to-SQL	Char/semantic NL trigger; T5-Base Comment: 0.44% poison, 79.41% ASR	TDW	–	R+L+S+X	GEN
SABER [72]	arXiv'24	CoT code generation	Self-attention trigger in reasoning steps	TDW	–	R+L+X	GEN
RTL-Breaker [96]	DATE'25	Verilog / HDL gen	Comment trigger → malicious HDL	TDW	A	R+C+X	GEN
Double Backdoored [52]	arXiv'24	Instruction-tuned LLMs	Malicious snippets in instruction data	TDW	–	R+L+X	GEN
FDI (continual learning) [142]	ISSTA'24	Completion	Feedback samples poison continual model updates	TDW	–	R	GEN
TrojanedCM [59]	arXiv'23	Defect, text-to-code	Pre-poisoned model repository	MW	A	R	MIX
STAB [16]	AAAI'26	Name prediction; summarization	Sharpness-aware transferable triggers	TDW	A	R	GEN
Sleeper Agents [56]	arXiv'24	Insecure code generation	Conditional insecure-code behavior under safety training	TDW	–	R+S	GEN

Most training attacks write data; You Autocomplete Me also alters weights, while TrojanedCM distributes a modified model. Targets now extend from labels, rankings, and summaries to insecure completions, SQL, HDL, and reasoning traces.

4.2.1 From Fixed Triggers to Distributed Payloads. Dead code, rare identifiers, and syntax-screened triggers control summaries, search rankings, and task predictions [82, 122, 140, 155]. Completion attacks instead associate contexts with insecure suggestions or unsafe natural-language–code pairs [27, 129]. TrojanPuzzle distributes placeholder associations; CodeBreaker obfuscates payloads against detectors; the generalized attack learns trigger-group mappings; and AFRAIDOOR adapts identifiers to the victim and detector [2, 125, 170, 176]. These mechanisms change the trigger specificity and feedback available to the attacker.

4.2.2 Expansion Across Tasks and Representations. Code-search backdoors target rankings, while summarization attacks use fixed, task-agnostic, low-rate, or transformation triggers [85, 116, 122, 156, 179]. SABER targets reasoning traces, ToxicSQL database queries, and RTL-Breaker Verilog [72, 88, 96]. Their oracles must assess the corresponding artifact. TrojanedCM's model-distribution setting additionally changes the required write access [59].

4.2.3 Trigger Validity and Outcome Evidence. Input preservation and payload success concern different objects. A trigger-bearing program may be intended to retain its behavior while the generated payload is deliberately harmful. CodePoisoner permits functionality changes and checks syntactic validity with tree-sitter; human assessment concerns naturalness [82].

Execution also serves different purposes. TrojanPuzzle, the generalized attack, and Double Backdoored test functional utility; CodeBreaker manually inspects payload functionality [2, 52, 125, 170]. ToxicSQL includes successful execution of target-bearing SQL in its ASR definition [88]. These are respectively checks of clean utility, payload validity, and executed attack queries.

4.2.4 Access, Budget, Persistence, and Adaptive Evaluation. Training budgets comprise poison counts, corpus proportions, filtering, schedules, and compute. Persistence may mean

held-out success, survival after filtering or clean fine-tuning, or transfer across datasets and checkpoints. The transferable backdoor study tests cross-dataset behavior [16]; CodeBreaker, AFRAIDOOR, and the generalized attack evaluate defense pressure. Sleeper Agents uses CodeQL to assess conditional insecure generation after safety training [56].

Findings. Training compromise can persist after malicious records are removed because the association resides in learned state. Trigger specificity, clean utility, and survival after filtering or fine-tuning determine what a low poison rate establishes (Table 23).

4.3 How Is Runtime Context Used to Redirect Generation or Agent Actions?

Runtime attacks place material in feedback stores, retrieval corpora, repositories, dependencies, skills, or tool channels that a deployed system later consumes. A stored artifact can influence many requests without changing model parameters. The control point lies in context ingestion, interpretation, or tool authority.

Table 9. Runtime-context attacks. Access combines context-write capability (RCW) and model knowledge (WB/GB/BB); *O* and *I* denote outcome evidence and result type.

Method	Venue'Yr	Context	Mechanism	Acc	O	I
FDI [142]	ISSTA'24	RAG codegen service	Feedback / retrieval-corpus injection	RCW+BB	R	GEN
ShadowCode [173]	TDSC EA'26	Completion	RAG / dependency prompt injection	RCW+ WB/ GB/BB	R	GEN
RACG KB poison [87]	TSE'26	RAG codegen	Vulnerable example planted in the retrieval corpus redirects generated code	RCW+BB	R+L	GEN
DrainCode [159]	ASE'25	RAG codegen	Corpus trigger inflates output length; +49% energy	RCW+ WB/BB	R+L+X	GEN
Your AI, My Shell [91]	arXiv'25	agent-injection	AIShellJack (314 payloads); command-exec success up to 84%	RCW+BB	R+L+X	AGT
Red-Teaming Agents [166]	arXiv'25	agent-injection	Tool-invocation RCE on 19/25 agent–LLM combinations	RCW+BB	R+X+E	AGT
SkillJect [69]	arXiv'26	Coding-agent skills	Malicious skill metadata and instructions induce tool actions	RCW+BB	R+X	AGT

Table 9 covers four retrieval or feedback attacks and three tool- or skill-based attacks. Their budgets use different units: a retrieval example, a shell payload collection, or a reusable skill, together with the number of later opportunities for consumption.

4.3.1 Retrieved Context and Feedback. FDI follows planted feedback to generated responses [142]. RACG judges output vulnerability with DeepSeek and manual validation [87]. ShadowCode measures target functionality and insertion position, but the inspected version leaves the assessor procedure unspecified [173]. DrainCode targets output length and energy use, reporting resource measurements separately from functional utility [159]. DrainCode has a local construction argument; ShadowCode's broader output-content variants extend beyond its nonfunctional-comment construction. Retrieval determines whether each stored record receives an opportunity to influence generation.

4.3.2 Repository, Skill, and Tool Injection. Your AI, My Shell and Red-Teaming Agents trace injected context to commands or remote code execution [91, 166]. SkillJect targets reusable skill metadata and instructions, affecting selection and later tool actions [69]. Tool permissions and the execution environment determine the reachable effects. Section 5 relates these paths to evaluated controls.

4.3.3 Relation to General-Agent and Supply-Chain Studies. Indirect prompt injection and AgentDojo supply broader attack and evaluation settings; Spotlighting and CaMeL address provenance and control/data separation [28, 29, 39, 51]. Related studies examine poisoned MCP descriptions, model-hub artifacts, and web-scale training writes [13, 161, 192]. Their entry points depend on whether the victim consumes an artifact as runtime context or uses it to update learned state. General-agent studies without a separate code-task evaluation inform the discussion but remain outside primary counts [95].

4.3.4 From Context Selection to Execution. A runtime attack proceeds through artifact selection, model interpretation, action proposal, authorization, and execution. An injected instruction can be ignored, a proposed action denied, or its effect contained. Section 6.1.2 distinguishes the reached stages; Table 12 aligns them with controls.

4.4 What Output, Privacy, and Ecosystem Risks Are Measured?

Risk studies measure insecure generation, privacy exposure, and ecosystem properties (Table 10). Ordinary generation estimates prevalence; optimized vulnerability-inducing requests are input attacks. Diagnostic privacy probes measure an existing training-data relationship.

Table 10. Output/ecosystem risk studies. *O* records categorical outcome evidence and may contain multiple codes. *I* records the evaluated result type (PRED/GEN/MIX/AGT/OBS).

Work	Venue'Yr	Area	Finding	O	I
Insecure generation and security-related code errors					
Asleep at the Keyboard [112]	S&P'22	insecure-gen	Approximately 40% of 1,689 programs; 89 security scenarios	L+C+S	GEN
Perry et al. [114]	CCS'23	insecure-gen	47 participants; manual security and correctness ratings	L	GEN
Lost at C [127]	USENIX Sec.'23	insecure-gen	58 C-program submissions; compilation, functional tests and manual security review	L+C+X	GEN
Fu et al. [35]	TOSEM'25	insecure-gen	733 assistant-attributed files; CodeQL findings and manual validation	L+S	OBS
Khoury et al. [74]	SMC'23	insecure-gen	Only 5/21 ChatGPT programs secure on first attempt	L+C+X	GEN
Hamer et al. [45]	SPW'24	insecure-gen	248 vs. 302 CodeQL findings in matched ChatGPT/Stack Overflow Java code	S	GEN
Mousavi et al. [99]	AsiaCCS'24	insecure-gen	Frequent Java crypto/security-API misuse	L+C+S	GEN
Large-scale GitHub [128]	ICICS'25	insecure-gen	4,241 CWE occurrences; inconsistent file denominators across reported sections	C+S	OBS
BaxBench [152]	ICML'25	insecure-gen	392 backend tasks with functional tests and end-to-end exploits	X+E	GEN
SecRepo Bench [132]	arXiv'25	insecure-gen	318 tasks in 27 C/C++ repositories; developer tests and OSS-Fuzz PoC crashes	C+X+E	GEN
Yan et al. [168]	arXiv'25	insecure-gen	Vanilla-prompt generation evaluated with CodeQL before repair	S	GEN

continued on next page

Table 10 (continued)

Work	Venue'Yr	Area	Finding	O	I
RealSec-bench generation measurement [160]	arXiv'26	insecure-gen	105 Java tasks in 30 repositories; tests, CodeQL and judged findings	L+S+X	GEN
SecureVibeBench agent-output study [18]	arXiv'25	insecure-gen	SecureVibeBench v5: 105 C/C++ tasks in 41 repositories; tests and historical PoVs	L+C+ S+X+E	GEN
LLM-CSEC [130]	arXiv'25	insecure-gen	84 C/C++ scenarios evaluated with three static tools	S	GEN
ChatGPT web-application pilot [64]	ACMSE'24	insecure-gen	Manual web-code security inspection and security-aware prompting	L	GEN
Commercial model study [10]	TETCI'25	insecure-gen	Dated author artifact separates functionality, security execution and lexical CWE recognition	L+X+ E	GEN
Practical hallucinations: security cases [191]	arXiv'24	insecure-gen	Unsafe YAML-loading case within a broader hallucination taxonomy	L	GEN
Memorization, membership, and leakage					
Unveiling (code) [178]	ICSE'24	memorization	40,125 memorized snippets from CodeParrot	R+L	GEN
Copilot recitation [196]	Blog'21	memorization	∼0.009% suggestions verbatim recitation	L	GEN
Traces of mem. [3]	ICSE'24	membership	Code LLMs vulnerable to extraction + MIA	R+L	GEN
Gotcha [177]	TSE'24	membership	Surrogate MIA: TPR 0.95 / FPR 0.10	R	PRED
AdvPrompt-MIA [71]	ASE'25	membership	Adversarial-prompt MIA, AUC >0.91 on HumanEval	R	PRED
Buzzer [188]	Findings EMNLP'24	membership	Detects unauthorized code use in CPLMs	R	PRED
CodexLeaks [110]	USENIX Sec.'23	leakage	∼8% of prompts leak PII from Codex	R+L	GEN
Credential Revealer [55]	FSE'24	leakage	2,702 plausible secrets; 200 repository matches; two operational public Stripe example keys	R+L+ X	GEN
License infringement [73]	FORGE'24	leakage	38M exact copyleft duplicates in datasets used by 53 code LLMs	L	OBS
DP code-model study [14]	arXiv'25	privacy	Code memorization and private fine-tuning with executed utility checks	R+L	GEN
NOIR reconstruction measurement [105]	arXiv'26	privacy	Cloud-side reconstruction under split code generation	R+L+ X	GEN
CodeMI [154]	arXiv'24	privacy	Token-rank shadow-model membership inference; performance varies by victim	R	PRED
Package ecosystem					
Package hallucination [136]	USENIX Sec.'25	package risk	440,445 of 2.23M package recommendations (19.7%) were hallucinated; 205k unique names	R+L	GEN

The counting units differ across this group: a vulnerable program, reproduced span, membership decision, valid credential, and absent package name represent distinct outcomes. Controlled prompts, repository observations, and service telemetry also sample different populations.

4.4.1 Insecure Code Generation. Asleep at the Keyboard reports vulnerabilities in roughly 40% of 1,689 Copilot programs; Khoury et al. find few secure programs among 21 initial ChatGPT responses [74, 112]. Later studies examine assistant snippets, Stack Overflow

comparisons, Java APIs, and AI-attributed GitHub files [35, 45, 99, 128]. Repository observations broaden deployment coverage but add attribution and selection uncertainty.

Static findings depend on analyzer rules and build context and can include unreachable weaknesses. User studies additionally measure developer behavior: Perry et al. observe less secure code and higher confidence, while Sandoval et al. find no significant increase in critical C bugs under a different task and participant design [114, 127].

BaxBench pairs backend functionality tests with executable exploits; SecRepoBench combines repository tasks, developer tests, and security outcomes [132, 152]. RealSec-bench adds Java projects, CodeQL, and adjudication, while SecureAgentBench, renamed SecureVibeBench before the cutoff, evaluates C/C++ projects with functional tests, historical proof-of-vulnerability cases, and static findings [18, 160]. Yan et al. measure ordinary-generation weaknesses with CodeQL before testing repair [168]. The broader hallucination taxonomy supplies an unsafe YAML-loading case but no separate security-prevalence rate [191].

BaxBench's OpenHands comparison uses 351 of 392 tasks, excluding Python-Django and the Compiler scenario [152]. Black et al.'s author artifact separates basic operation, fuller functionality, security tests, and lexical CWE mentions at its 29 April 2024 commit [10]. These retained subsets and channels determine the corresponding comparisons.

4.4.2 Memorization, Membership, and Leakage. Memorization measures reproduction of training-like material, membership inference classifies candidate inclusion, and extraction recovers content. Copilot recitation and CodeParrot use different prompts, matching thresholds, corpus knowledge, and denominators [178, 196]. Exact matches favor precision; fuzzy matches capture transformed code with weaker provenance attribution.

Traces of Memorisation tests prefix-conditioned extraction; Gotcha, AdvPrompt-MIA, and Buzzer assess membership [3, 71, 177, 188]. Membership AUC, TPR, and FPR depend on candidate sampling, duplicates, temporal splits, and contamination. CodeMI uses token ranks and shadow training, with attack performance varying across recurrent and transformer victims [154].

CodexLeaks measures generated PII. Credential Revealer distinguishes plausible secrets, repository matches, and two operational keys from public Stripe examples [55, 110]. Public example-key validity provides no evidence of private payment-credential disclosure. DP-Code measures memorization and private fine-tuning, while NOIR studies cloud-side reconstruction during split generation [14, 105]. Dataset-license overlap concerns exact duplicates in corpora rather than observed model disclosure [73].

4.4.3 Package Ecosystem. Spracklen et al. find 440,445 hallucinated package names among 2.23 million recommendations (19.7%), drawn from 576,000 generated samples [136]. Recommendation counts measure encounter frequency; unique names measure exposed namespace. Registry absence depends on the dated snapshot, normalization, scopes, and aliases. Attempted use, attacker registration, and malicious installation are subsequent events with separate denominators.

Findings. These measurements locate exposure at different stages: code generation, developer use, training-data inference, disclosure, or package recommendation. Their sampling procedure and counting unit determine how each result applies to deployment.

5 RQ3: What Defenses Have Been Evaluated for Neural Code Systems?

The 110 defense contributions comprise 38 input-robustness, 29 training-compromise, 4 runtime-context, and 39 output, privacy, or ecosystem evaluations. Table 11 includes mitigations evaluated within attack and risk papers; some assess utility without measuring attack reduction.

Table 11. Defenses by protection target. *PP* denotes preservation support; *O*, outcome evidence; and *I*, result type. Codes are defined in Sections 2.5, 6.1.1, and 6.1.2.

Method	Venue'Yr	Mechanism	PP	O	I
Robustness to inference-input attacks					
MHM [184]	AAAI'20	Adversarial training (rename)	A+T	R+C+X	PRED
Semantic Robustness [50]	SANER'22	Adversarial training (minimax)	A	R	GEN
ALERT [174]	ICSE'22	Adversarial fine-tuning	A	R	PRED
CARROT [183]	TOSEM'22	Adversarial training (valid transforms)	A	R	PRED
RoPGen [86]	ICSE'22	Data and gradient augmentation	A	R	PRED
ACCENT [195]	TOSEM'22	Identifier masking and augmentation comparisons	A	R+L	GEN
CODA [147]	ASE'23	Adversarial fine-tuning	A	R	PRED
CodeFort [189]	Findings EMNLP'24	Robust training (logit pairing, contrastive)	A	R+X	GEN
GenCode [32]	EMSE'26	Generation-and-selection augmentation for code understanding	U	R	PRED
MixCode [31]	SANER'23	Mixup + refactoring augmentation	A	R	PRED
ContraCode [63]	EMNLP'21	Contrastive pretraining (compiler variants)	U	R	MIX
CLAWSAT [67]	SANER'23	Contrastive + adversarial training	A	R	MIX
ContraBERT [90]	ICSE'23	Contrastive pretraining (9 operators)	U	R	MIX
DISCO [30]	ACL'22	Structure-aware contrastive pretraining	A	R	PRED
Behavior-preserving aug. [117]	SecureComm '24	Augmentation (vuln detection)	A	R+L+C	PRED
ENBECOME [46]	arXiv'26	**Certified** (randomized smoothing, L_0)	A	R+F	PRED
EP-Shield [101]	arXiv'25	Detection + purification (naturalness)	A	R	MIX
CodeGarrison [38]	JSS'25	Embedding classifier for adversarially perturbed code	–	R	PRED
CoCoFuzzing retraining [162]	IEEE Trans. Reliab.'23	Augmentation with neuron-coverage-selected variants	A	R	GEN
CoTR-D [171]	IST'25	Distance-based augmentation and adversarial training	A	R+X	GEN
FuncFooler retraining [68]	arXiv'22	Adversarial augmentation of Asm2Vec training data	A	R	PRED
PELICAN hardening [190]	USENIX Sec.'23	Adversarial training with trigger-injected binaries	A+F	R	PRED
ITGen fine-tuning [54]	ICSE'25	Adversarial augmentation and fine-tuning	A	R	MIX
CodeCrash prompt control [76]	NeurIPS'25	Instruction to ignore comments; effectiveness varies by setting	–	R	GEN
DAMP controls [181]	OOPSLA'20	Normalization, outlier detection, and adversarial training	A	R	PRED
Abstention/ refinement [9]	ICML'20	Adversarial training, abstention, and representation refinement	A	R	PRED
STRATA controls [137]	arXiv'20	Evaluation of a robustly trained checkpoint	A	R	GEN
Optimized obfuscation [138]	ICLR'21	Adversarial training on optimized code obfuscations	A	R	GEN
Mutation-guided training [151]	ICST'21	Augmentation with mutation-generated examples	A	R	MIX

continued on next page

Table 11 (continued)

Method	Venue'Yr	Mechanism	PP	O	I
DIP training [102]	ACL'23	Adversarial retraining using individual and combined attacks	A	R	PRED
RNNS training [187]	EMNLP-F'23	Contrastive adversarial retraining	A	R	PRED
Structural training [22]	EMNLP-F'23	Adversarial training on transformed AST representations	U	R	MIX
Transfer prompt controls [182]	ASE'24	Few-shot, inverse-transformation, and meta-prompt controls	A	R	PRED
NatGVD controls [123]	arXiv'25	Ensembles and adversarial retraining	A	R	PRED
GraphCodeAttack adversarial retraining [107]	arXiv'23	Adversarial retraining using graph-pattern insertions	A	R	PRED
DegradePrompter guided prompting [53]	arXiv'24	Guiding prompts evaluated on attacked synthesis requests	–	R+X	GEN
MOAA adversarial fine-tuning [193]	FSE'24	Adversarial fine-tuning on defect, clone and authorship classification	A	R	PRED
INSEC filtering controls [65]	ICML'25	Perplexity filtering and constrained adaptive attack tests	–	R+X	MIX
Poisoning and backdoor defenses					
Spectral sig. (code) [122]	ICPR'22	Detection (spectral signatures)	–	R	MIX
SS eval (code search) [155]	FSE'22	Detection (SS weak for code)	–	R	PRED
SS+AC eval [129]	USENIX Sec.'21	Detection (largely ineffective)	–	R	MIX
CodeDetector [82]	TOSEM'24	Detection (integrated-gradients probing)	–	R+C	MIX
EliBadCode [139]	FSE'25	Trigger inversion + unlearning	A	R	PRED
OSeqL [58]	CAIN'25	Detection (line occlusion)	–	R	PRED
Trojan signatures [60]	arXiv'24	Detection (negative result for code)	–	R	PRED
IDBA [118]	IST'25	Detection (sticker variants; particle filtering)	A	R	PRED
KillBadCode [141]	ICSE'25	Detection (code-naturalness perplexity)	–	R	MIX
DePA [150]	arXiv'25	Dead-code detection and localization	–	R	PRED
CodePurify [100]	arXiv'24	Trigger detection and mask-infilling purification	A	R	MIX
MT4DP [17]	arXiv'25	Detection (metamorphic testing)	–	R	PRED
FDI screening [142]	ISSTA'24	Activation clustering, spectral signatures, and ONION	–	R	PRED
SET controls [179]	arXiv'25	Screening and code-normalization comparisons	A	R	MIX
TrojanPuzzle controls [2]	S&P'24	Fine-pruning with clean-utility evaluation	–	R+S+X	MIX
CodeBreaker controls [170]	USENIX Sec.'24	Static, model-based, and representation-based screening	–	R+S	PRED
Generalized-attack controls [125]	AsiaCCS'25	Static filtering, deduplication, spectral screening, and pruning	–	R+S+X	MIX
BADCODE controls [140]	ACL'23	Activation clustering and spectral-signature screening	–	R	PRED
BadCS controls [116]	arXiv'23	Spectral-signature and keyword screening	–	R	PRED
AFRAIDOOR controls [176]	TSE'24	Spectral, activation-clustering, and token-filtering controls	–	R	MIX
Multi-target repair [85]	ACL'23	Fine-pruning and weight reinitialization	–	R	MIX
ToxicSQL controls [88]	SIGMOD'26	SQL screening and natural-language input augmentation	–	R+L+S+X	MIX

continued on next page

Table 11 (continued)

Method	Venue'Yr	Mechanism	PP	O	I
STAB controls [16]	AAAI'26	Spectral signatures, ONION, and Kill-BadCode	–	R	MIX
Low-rate backdoor controls [156]	ASE'25	Spectral-signature screening at low poison rates	–	R	PRED
SABER ONION screening [72]	arXiv'24	ONION screening measured against reasoning-step backdoors	–	R	PRED
RTL comment-removal utility study [96]	DATE'25	Comment-removal generation-utility study; removal ASR not measured	A	C+X	GEN
CodeScan [169]	arXiv'26	Vulnerability-oriented black-box scanning of poisoning-sensitive generation	–	R+L	MIX
Sleeper Agents code safety training [56]	arXiv'24	Safety-training controls for conditional insecure-code behavior	–	R+S	GEN
Code spectral-signature study [77]	arXiv'25	Spectral-signature screening and filter/retrain evaluation	–	R	MIX
Runtime-context defenses					
SkillJect controls [69]	arXiv'26	Prompt hardening and skill-scanner evaluation	–	R+X	AGT
TIP guard model [166]	arXiv'25	Prompt-Guard screening of malicious tool descriptions	–	R	PRED
DrainCode controls [159]	ASE'25	SVM, perplexity, and CodeBERT context screening	–	R	PRED
ShadowCode controls [173]	arXiv'24	Context removal and embedding-based screening	U	R	MIX
Secure-generation and ecosystem defenses					
SVEN [47]	CCS'23	Security-steering prefix vectors	–	C+S+X	GEN
SafeCoder [48]	ICML'24	Security instruction tuning	–	R+S+X	GEN
CodeGuard+ [34]	arXiv'24	Constrained secure decoding	–	S+X	GEN
PromSec [103]	CCS'24	gGAN + LLM prompt-optimization loop	T	R+S+X	GEN
Scan-and-repair [168]	arXiv'25	Scanner-feedback repair loop	–	S	GEN
CodeShield [97]	Meta'24	Inference guardrail for insecure code	–	R+S	PRED
Copilot Chat repair [35]	TOSEM'25	Prompted repair followed by repeated static analysis	–	L+S	GEN
ChatGPT repair [74]	SMC'23	Security feedback and prompted correction	–	L+C+X	GEN
Package-hallucination mitigation [136]	USENIX Sec.'25	Package checking, refinement, retrieval and fine-tuning; utility measured separately	–	R+L+X	MIX
BaxBench security reminders [152]	ICML'25	Generic security reminder and scenario-CWE oracle reminder	–	X+E	GEN
RealSec-bench security prompting [160]	arXiv'26	Security guidelines evaluated on Java repository tasks	–	L+S+X	GEN
SecureVibeBench security reminders [18]	arXiv'25	Security reminders evaluated in the original v1 agent benchmark	–	C+S+X+E	GEN
SPARK [167]	arXiv'26	Security knowledge priming and representation-guided activation	–	R+L+S+X	GEN
SCS-Code [163]	arXiv'26	Security concept steering through internal representations	–	R+S+X	GEN
LLM-CSEC security system prompt [130]	arXiv'25	Security system prompt evaluated with C/C++ static tools	–	S	GEN
Prompting-technique security study [148]	arXiv'24	Security prompting compared on generated Python code	–	L+S	GEN
ChatGPT web-application security prompts [64]	ACMSE'24	Security-aware prompting for a web-application case study	–	L	GEN

continued on next page

Table 11 (continued)

Method	Venue'Yr	Mechanism	PP	O	I
Commercial-model prompt guidance [10]	TETCI'25	Security prompting; functional and security execution in author artifact	–	L+X+E	GEN
CodeMirage hallucination detection [1]	arXiv'24	Classifier for hallucination types including security errors	–	R+L	PRED
Privacy defenses					
PrivCode [92]	NDSS'26	Differentially private code synthesis	–	R+L+C+X+F	GEN
PrivCode++ [93]	ICML'26	Differentially private prompt–code synthesis	–	R+L+X+F	GEN
Gotcha decoding mitigation [177]	TSE'24	Top-k sampling compared with beam search for membership leakage	–	R	PRED
Credential suggestion blocking [55]	FSE'24	Public-code suggestion blocking; lexical secret exposure measured	–	L	GEN
DP code-model fine-tuning [14]	arXiv'25	DP-SGD code fine-tuning and empirical disclosure evaluation	–	R+L+X+F	GEN
NOIR [105]	arXiv'26	Split generation with a conditional inference-channel privacy guarantee	–	R+L+X+F	GEN
Provenance and unlearning					
CodeMark [143]	FSE'23	Repository watermark through transformation triggers	A	R+L	MIX
CoProtector [144]	WWW'22	Repository poison/watermark for code-model protection	–	R	MIX
SWEET [79]	ACL'24	Watermark (high-entropy tokens)	U	R+X+F	MIX
CodeIP [41]	Findings EMNLP'24	Watermark (grammar-guided multi-bit)	–	R+L	MIX
ACW [80]	arXiv'25	Watermark (behavior-preserving transforms)	A+T	R+X	PRED
DetectCodeGPT [133]	ICSE'25	AI-code detection (naturalness perturbation)	A	R	PRED
GPTSniffer [106]	JSS'24	AI-code detection (CodeBERT classifier)	–	R	PRED
Rewrite-detect [180]	AAAI'25	AI-code detection (rewrite self-consistency)	U	R	PRED
CodeMirage [43]	arXiv'25	AI-code detection benchmark (10 langs)	U	R+L	PRED
Scrub It Out [26]	ICSE'26	Unlearning (erase memorized secrets)	–	R+L+X	GEN
LLMs4Code unlearn [40]	Neural Netw.'26	Unlearning (scrub PII, retain utility)	–	R+L+X	GEN
ModMark [186]	arXiv'24	Tokenizer-level model watermark; full transformation validity unresolved	U	R	GEN
PROD [70]	arXiv'25	Source-code unlearning; privacy, security and functional utility assessed	–	R+L+S+X	GEN
FunPoison [165]	arXiv'26	Owner-side function protection with shared-input behavior checks	T	R+C+S+X	MIX

5.1 How Is Robustness to Input Manipulation Improved?

Adversarial training reduces sensitivity to selected input transformations. MHM, Semantic Robustness, ALERT, CARROT, CODA, and CodeFort vary the generator and training objective [50, 147, 174, 183, 184, 189]. GenCode selects generated examples, MixCode combines mixup and refactoring, and ACCENT compares masking with augmentation [31, 32, 195]. Many attack papers also evaluate retraining (Table 11).

Contrastive methods align compiler-derived, source-transformed, or adversarial views [30, 63, 67, 90]. The complete operator set determines the retained property: ContraCode subsamples lines, ContraBERT deletes or duplicates text, and GenCode includes syntax-breaking operations. RoPGen combines data and gradient augmentation for authorship

attribution, while EP-Shield detects and purifies inputs using naturalness [86, 101]. CodeGarrison classifies adversarial inputs through learned embeddings [38].

ENBECOME uses randomized smoothing to certify prediction stability under an identifier-level L_0 threat [46]. Its certified radius applies to that perturbation space; preservation and clean utility require their own checks.

5.2 How Are Poisoning and Backdoors Detected or Removed?

Backdoor defenses screen data, probe trained models, purify inputs, or remove learned associations. Spectral signatures and activation clustering often weaken when poisoned code resembles clean data; trojan-signature evaluation also reports a negative result [60, 122, 129, 155].

Code-specific detectors use integrated gradients, line occlusion, or sticker variants with particle filtering [58, 82, 118]. IDBA diagnoses behavior through sticker variants and returns a warning or original prediction. CodePurify replaces suspected trigger elements and evaluates the resulting task input [100]. Naturalness-based screening and localization can reject unusual clean code; purification can change its behavior [141, 150].

MT4DP tests natural-language search queries through metamorphic relations [17]. EliBadCode combines trigger inversion and unlearning, including an adaptive evaluation [139]. Signature tests filter-and-retrain controls; CodeScan probes generators with vulnerability-oriented queries [77, 169]. Detection scores measure discrimination, whereas removal requires evaluating residual triggered behavior. Clean removals, near-trigger false positives, and attacker adaptation determine the operating trade-off.

5.3 How Is Generated Code Made More Secure?

SVEN uses learned prefixes, SafeCoder security-oriented tuning, and CodeGuard+ constrained decoding [34, 47, 48]. SCS steers residual representations, while SPARK combines security priming and activation control [163, 167]. Prompting evaluations span controlled Python tasks, C/C++ static checks, and a manually assessed web case [64, 130, 148].

PromSec optimizes prompts, scanner-feedback systems request repairs, and CodeShield filters suggestions [97, 103, 168]. Their coverage follows the analyzer and repair or rejection policy. CodeShield's April 2024 deployment quantity counts screened suggestions. Fu et al. test Copilot Chat repairs; Khoury et al. correct seven of 16 initially insecure programs after feedback [35, 74]. Yan et al.'s June 2025 repair assessment uses CodeQL; its quoted HumanEval and MBPP values come from other studies and do not test the repaired outputs [168].

5.4 How Are Privacy and Code Provenance Protected?

DP-Code applies private fine-tuning; PrivCode synthesizes through a private junior model and post-processing; PrivCode++ protects jointly sensitive prompt–code pairs [14, 92, 93]. Their guarantees depend on adjacency and privacy accounting. NOIR protects a split-generation inference channel under a cryptographic threat model [105]. Functional tests assess retained utility.

CodeMark embeds repository transformation signals; CoProtector deliberately changes functionality to discourage unauthorized training; ModMark modifies tokenization [143, 144, 186]. SWEET, CodeIP, and ACW watermark generated code [41, 79, 80]. CodeIP's utility evidence comprises CodeBLEU and message recovery, without executable task tests.

Origin detectors use naturalness perturbations, classifiers, or rewrite consistency; CodeMirage tests ten languages and adapted generators [43, 106, 133, 180]. Scrub It Out and

LLMs4Code unlearning target memorized information [26, 40]; ProD assesses source-code unlearning, functionality, and security [70]. Gu et al.'s controlled test uses fictional biographies and measures judged leakage alongside pass@1. FunPoison instead validates transformed Java functions against shared outputs before measuring owner-side protection [165].

Credential Revealer compares public-code blocking settings [55]. Blocking reduces plausible secrets and repository matches for Copilot, while CodeWhisperer counts change little. The comparison has no paired executable-utility assessment.

5.5 How Do Runtime Controls Constrain Context and Agent Actions?

ShadowCode and DrainCode screen or remove context, SkillJect tests prompt hardening and skill scanners, and the TIP study screens tool descriptions with a guard model [69, 159, 166, 173]. Table 12 maps their trust path to control families: provenance and control/data separation act on interpretation, capability scoping on authorization, and sandbox policies on execution. A common evaluation across these layers and injection families remains open.

6 RQ4: How Are Security and Robustness Evaluated?

We examine preservation checks and outcome oracles, then resources, execution settings, and comparison protocols.

6.1 Which Preservation and Outcome Checks Are Reported?

Each contribution has an outcome-evaluation field. Contributions that transform an existing program while retaining task-relevant behavior also have a preservation field (Table 13).

6.1.1 Program-Preservation Support. A preservation contract specifies the retained behavior or label, assumptions, and scope. It may concern benchmark outputs, a binary block's register state, or a diagnostic sticker. Feature edits additionally require realizability as a program with the retained property. The multi-label support codes are:

- **A** A stated transformation rule or construction argument.
- **T** Original/transformed checks under shared inputs or an expected-output oracle.
- **F** Formal equivalence analysis under a specified operational model.
- **U** A required premise lacks support for the complete coded transformation family.
- **–** No preservation-dependent transformation is coded.

The contract can exclude the security behavior being changed, as in repair, or preserve an already malicious payload during obfuscation. We record test coverage over accepted variants, samples, or unspecified subsets. Parsing and compilation are separate fields; unchecked entries remain distinct from checks absent in the inspected protocol. Applicability and source scope are retained in the companion data.

6.1.2 Outcome Evidence. Outcome codes identify the assessment procedure: R for a model or task metric; L for lexical/statistical matching or human/LLM judgment; C for parsing, typing, or compilation; S for static program analysis; X for execution or tests; E for a demonstrated security or availability effect; and F for an outcome guarantee. Codes can overlap. A rate derived from an analyzer or test is counted under that oracle; R denotes a separately extracted scoring channel. An LLM vulnerability judgment is L. Unresolved oracles receive NR and contribute to no evidence-channel total.

Execution purpose distinguishes clean utility, payload testing, and agent actions. HumanEval on a poisoned model checks clean utility; a logged agent command supports X. An

Table 12. A code-agent control stack aligned with the runtime-context trust path. Representative records indicate where the included attacks exercise each stage; controls may act at more than one stage.

Trust-path stage	Evaluation question	Representative records	Control families
Write and ingestion	Can untrusted content enter a repository, index, skill, or tool channel?	FDI, ShadowCode, RACG poison, DrainCode, and SkillJect	Authenticated sources, signed artifacts, write access control, ingestion scanning, and quarantine
Selection and retrieval	Is the malicious artifact selected and placed in model-visible context?	FDI, ShadowCode, RACG poison, DrainCode, and SkillJect	Source allowlists, retrieval restriction, provenance-aware ranking, isolation, and taint labels
Interpretation	Does the model treat untrusted data as an instruction?	All records in Table 9, with different terminal goals	Instruction hierarchy, provenance marking, typed control/data separation, and context compartmentalization
Action proposal	Does the model propose a harmful completion, command, or tool call?	Your AI My Shell, Red-Teaming Agents, and SkillJect	Output policies, structured tool schemas, argument validation, and plan inspection
Authorization	Is the proposed action permitted under the agent's identity and policy?	Your AI My Shell, Red-Teaming Agents, and SkillJect	Capability scoping, least privilege, credential isolation, and risk-based human approval
Execution	Does the environment carry out the action with relevant resources?	Your AI My Shell, Red-Teaming Agents, and SkillJect	Sandboxing, filesystem and process isolation, network egress control, quotas, and timeouts
Terminal effect	Does execution produce the claimed security or availability consequence?	TIP hijacking case studies	Monitoring, audit traces, secret redaction, rollback, incident response, and outcome-specific oracles

additional E requires an effect established by the oracle, such as remote execution, unauthorized state change, or client disruption. FDI measures retrieval-to-suggestion success and SkillJect helper execution [69, 142]; these endpoints determine their respective codes.

Table 13. Evaluation fields recorded from the primary studies.

Field	Recorded information
Program-preservation check	Whether the authors report a transformation rule and stated preconditions, parsing or compilation, execution comparing the original and transformed programs, or formal analysis under stated assumptions. This field applies only to transformation-based studies.
Outcome evaluation	Whether the authors report a model- or task-level metric, lexical or statistical analysis, parsing or compilation, static analysis, dynamic execution or tests, an end-to-end effect, or a formal guarantee.

Table 14. Primary-source check coverage by role. Protocol checks inspect the named version's evaluation; targeted checks concern the recorded claim.

Role	Records	Protocol	Targeted	Limited
Attacks	75	70	5	0
Risk studies	30	28	2	0
Defenses	110	104	6	0
Evaluation resources	4	3	1	0

Table 15. Program-preservation support by role. Tags can overlap.

Role	Applicable	U	A	T	F
Attacks	55	7	48	4	1
Risk studies	2	0	2	0	0
Defenses	49	9	38	4	1
Evaluation resources	1	0	1	0	0

AUTHOR TODO 04 / 作者待办：核验保持条件与结果证据

请优先核对原文版本、定位和测试范围：Quiring/MHM 的 A+T、PELICAN 的局部 A+F、PromSec 的 20 个特制案例；再确认 FDI 的 R、SkillJect 的 R+X 和 TIP 的 R+X+E。逐项区分原始/变换程序的配对检查、良性功能测试、恶意载荷测试及智能体执行，确认表中代码与正文解释一致，并记录决定。

6.1.3 Evidence Patterns in the Coded Corpus. Table 17 summarizes outcome-evidence occurrences by attack surface and study role. Codes are multi-label, so a row may contain more occurrences than records.

Model and task metrics dominate input and training attacks (Table 17). Runtime outcomes range from generated suggestions to logged actions and demonstrated effects. RedCode also observes environmental effects after actions induced through the current request [42].

Table 16. Examples of preservation contracts and security outcome oracles in the included studies.

Study	Source location	Property established by the reported check
CodeCrash [76]	Sec. 3.2 and Fig. 1	A+T: each perturbed snippet must retain the task output on benchmark inputs before model evaluation.
CoTR [171]	Secs. 4.1 and 6.2	A+T: generated variants are tested, with coverage denominator unspecified. Translated-output tests separately support X.
PELICAN [190]	Sec. 5, p. 2371	A+F: symbolic equivalence is checked locally for synthesized blocks under the modeled state.
ACW [80]	Version 4, Secs. IV-A and IV-D	A+T: original and watermarked code are assessed on shared benchmark tests.
FDI [142]	Secs. 5.1.5 and 7	R: success ends at the generated suggestion, before exploitation.
Your AI, My Shell [91]	Secs. 4.2–4.3, Algorithm 1	R+L+X: expected and logged commands are matched; the endpoint is command execution.
SkillJect [69]	Version 3, Sec. IV-A	R+X: success ends at execution of the hidden helper.
TIP hijacking [166]	Secs. 6.1–6.3	Outcome R+X+E: controlled cases report remote command execution and persistent client unresponsiveness.
Quiring et al. [121]	Transformation and testing protocol	A+T: candidate programs must retain output on the original contest inputs; human plausibility is assessed separately.
MHM [184]	p. 1174, before Table 4	A+T: restored source variants are compiled and executed against the supplied tests; the paper reports agreement with original results.
PromSec [103]	Sec. 5.2, Table 4	T: differential fuzzing compares outputs on 20 specially constructed codebases; this subset differs from the full security evaluation.
FunPoison [165]	Java validation protocol	T: shared expected outputs and project tests validate protected functions over the exercised inputs.

Table 17. Outcome evidence by attack surface and study role. Defenses are shown separately because they protect different surfaces.

Group	Records	R	L	C	S	X	E	F
Inference-input attacks	46	44	15	5	2	11	1	0
Training compromise	22	22	8	2	5	7	0	0
Runtime-context attacks	7	7	3	0	0	4	1	0
Output/ecosystem risk	30	11	20	7	9	9	4	0
Defenses	110	96	24	9	20	32	3	6

Construction arguments dominate preservation support (Table 15). Paired checks occur in Quiring et al., MHM, CodeCrash, CoTR, PromSec, ACW, and FunPoison; PELICAN provides local symbolic equivalence (Table 16). These checks cover different variants, inputs, outputs, and modeled state. PromSec's 20-case preservation test, for example, is a subset of its full security evaluation.

6.2 Which Datasets and Benchmarks Support the Evaluations?

Tables 18 and 19 separate security resources from underlying task datasets. Security resources add weakness-oriented prompts, transformations, execution environments, repositories, and exploit paths to conventional task labels and tests.

Table 18. Security and robustness benchmarks and toolkits, with their task and oracle coverage.

Name	Venue'Yr	Type	Content
SecurityEval [135]	MSR4P&S'22	security-bench	130 prompts across 75 CWE types
LLMSecEval [149]	MSR'23	security-bench	150 NL prompts, MITRE Top-25, secure references
CyberSecEval [8]	arXiv'23	security-bench	Insecure-code and attack-helpfulness evaluation
CyberSecEval 2 [7]	arXiv'24	security-bench	Prompt injection and interpreter abuse
CyberSecEval 3 [153]	arXiv'24	security-bench	Offensive-security operations and Llama 3 evaluation
CodeLMSec [44]	SaTML'24	security-bench	280 prompts eliciting vulnerabilities in black-box models
SALLM [134]	ASEW'24	toolkit	100 prompts with static and dynamic assessment
CodeSecEval [157]	arXiv'24	security-bench	180 samples across 44 CWEs for generation and repair
CWEval [113]	LLM4Code'25	security-bench	Joint functionality and security with execution oracles
SecCodePLT [108]	NeurIPS'25	security-bench	5.9k samples across 44 CWE categories
DiverseVul [24]	RAID'23	security-bench	18.9k vulnerable and 330k benign functions across 150 CWEs
CrossVul [109]	FSE'21	security-bench	Cross-language vulnerability and patch pairs with CVEs
CodeShield [97]	Meta'24	toolkit	Inference-time insecure-code guardrail
ReCode [158]	ACL'23	robustness-bench	More than 30 perturbations on HumanEval and MBPP
RobustNPR [37]	JSEP'24	robustness-bench	Robustness evaluation for neural program repair
Natural robust NPR [78]	TOSEM'25	robustness-bench	Naturalness-vetted program-repair robustness
BaxBench [152]	ICML'25	security-bench	392 backend tasks with functional tests and end-to-end exploits
SecRepoBench [132]	arXiv'25	security-bench	318 tasks in 27 C/C++ repositories with unit tests and OSS-Fuzz PoCs
RealSec-bench [160]	arXiv'26	security-bench	105 Java tasks in 30 repositories; tests, CodeQL and adjudicated findings
SecureVibeBench [18]	arXiv'25–26	security-bench	Pre-cutoff v5: 105 C/C++ tasks in 41 repositories; functional tests and historical PoVs

Table 19. Underlying task datasets commonly reused in neural code security and robustness evaluation.

Name	Venue'Yr	Type	Content
CodeXGLUE [94]	NeurIPS'21	task-dataset	14 datasets covering 10 code-intelligence tasks
CodeSearchNet [57]	arXiv'19	task-dataset	Approximately 6M functions in six languages for code search
BigCloneBench [145]	ICSME'15	task-dataset	Approximately 8M validated Java clone pairs
POJ-104 [98]	AAAI'16	task-dataset	52k C/C++ submissions for 104 problems

continued on next page

Table 19 (continued)

Name	Venue'Yr	Type	Content
Devign [194]	NeurIPS'19	task-dataset	27k labeled C functions from FFmpeg and QEMU
ReVeal [15]	TSE'22	task-dataset	Function-level vulnerability detection data
Draper VDISC [126]	ICMLA'18	task-dataset	1.27M C/C++ functions labeled by static analysis
Py150 [124]	OOPSLA'16	task-dataset	150k Python files for completion
Project CodeNet [115]	NeurIPS'21	task-dataset	Java250 and Python800 classification
HumanEval [20]	arXiv'21	task-dataset	164 Python problems with unit tests
MBPP [4]	arXiv'21	task-dataset	974 entry-level Python tasks
APPS [49]	NeurIPS'21	task-dataset	10k coding-challenge problems
CodeContests [84]	Science'22	task-dataset	Approximately 10k Codeforces-style problems
CONCODE [62]	EMNLP'18	task-dataset	Approximately 100k Java methods with natural-language context
Google Code Jam [11]	USENIX Sec.'15	task-dataset	Authorship-attribution corpus
DS-1000 [75]	ICML'23	task-dataset	1000 data-science code-generation problems

Fragment datasets such as BigCloneBench, CodeSearchNet, Devign, ReVeal, and CodeXGLUE provide labels or rankings but often require reconstruction for execution [15, 57, 94, 145, 194]. HumanEval, MBPP, APPS, CodeContests, POJ-104, and CodeNet provide runnable solutions or task tests in varying forms [4, 20, 49, 84, 98, 115].

SecurityEval, LLMSecEval, CodeLMSec, and CodeSecEval emphasize known weakness scenarios [44, 135, 149, 157]. CyberSecEval adds attack helpfulness, injection, interpreter abuse, and offensive-security behavior [7, 8, 153]. These resources trade breadth of tasks and models against the depth of executable security evidence.

6.3 What Do Executable and Agent Evaluations Establish?

Functional tests assess requested behavior; security tests exercise specified safety properties. SALLM and CWEval combine both [113, 134]. SecRepoBench runs developer tests and OSS-Fuzz PoCs within repositories; a PoC-triggered crash supports X+E for that input. BaxBench instead tests self-contained backends with executable exploits [132, 152].

Execution depends on dependencies, filesystem and network access, secrets, timeouts, and isolation. Agent traces further reveal whether proposed operations are ignored, denied, executed, or contained. These environmental conditions distinguish results obtained with similar model interfaces.

6.4 What Do Comparisons within the Same Study Reveal?

Table 20 compares defenses under shared victim, task, and evaluation settings within each study.

Protection depends on the evaluated attack family. MHM reduces ASR by 24.9 percentage points for LSTM and 37.4 for ASTNN; STRATA and DIP expose residual weaknesses under other transformations. Security gains can also reduce utility: package fine-tuning lowers hallucinated recommendations by 13.48 points while HumanEval pass@1 falls by 26.1 points.

SET-Backdoor illustrates trigger-specific protection. At 5% poisoning, its CodeBERT P8.2 baseline reports ASR/accuracy of 89.09/62.06. Related P8.3 normalization reduces

Table 20. Selected comparisons under a shared study protocol. ASR denotes attack success rate; all rates are percentages. F1 values remain on the source's 0–1 scale.

Study	Controlled comparison	Reported result	Interpretation
MHM [184]	Same architectures and evaluation, retrained with 2,000 adversarial examples (Table 4)	LSTM clean accuracy 92.9 to 94.0; ASR 71.3 to 46.4. ASTNN clean 98.2 to 98.1; ASR 92.1 to 54.7.	Training reduces the tested rename attack while leaving substantial residual success; clean accuracy changes little.
STRATA [137]	Original vs. robust checkpoint under gradient and STRATA attacks (Table 9)	Original clean/gradient/STRATA F1: 0.363/0.243/0.212. Robust: 0.367/0.342/0.240.	Protection transfers unevenly: the held-out STRATA attack retains a larger effect than the gradient attack.
DIP [102]	CodeBERT clone detector trained on MHM, ALERT, DIP, or all three (Table 3)	DIP ASR is 32.0/41.5/15.0/8.0 under those training sets. MHM ASR is 2.5/1.5/9.2/1.0.	A defense specialized to one family can leave another family effective; mixed training improves this tested coverage.
Package mitigation [136]	DeepSeek-Coder-6.7B before/after fine-tuning; generation and HumanEval evaluated separately	Hallucinated recommendations: 16.14 to 2.66. HumanEval pass@1: 51.4 to 25.3.	The reduction in package hallucination coincides with lower functional utility; a security score alone hides this operating cost.

ASR to 7.15, while unrelated P7.2 leaves it at 88.95 [179]. Held-out trigger families and retained utility are therefore central to evaluating normalization.

AUTHOR TODO 05 / 作者待办：核验数字对照及结论方向

请逐项对照原文和 within_study_comparisons.csv：MHM 的 2,000 条训练样本及 ASR/准确率；STRATA 的 0–1 F1；DIP 的训练集顺序和攻击行列；包幻觉的推荐次数分母及 HumanEval 变化。另核对 SET 的 5% 投毒与 P8.2/P8.3/P7.2 对照，确认百分点差值及安全/功能权衡的解释，并记录核验结果。

6.5 What Makes Results Comparable across Studies?

Table 21 identifies the settings that must be aligned for four representative comparisons.

Table 21. Examples of results that require matched re-evaluation before direct comparison.

Pair	Why the reported results differ	What a matched comparison would require
ALERT [174] vs. INSEC [65]	Both manipulate current inputs, but ALERT targets a discriminative prediction through a transformed program, whereas INSEC targets insecure generated code through an injected comment. Their preservation obligations and outcome oracles differ.	A common victim task, matched feedback and query budget, explicit transformation validity, and separate model, functional, and security outcomes.
TrojanPuzzle [2] vs. ShadowCode [173]	Both can induce insecure suggestions, but the former writes fine-tuning records and changes learned behavior; the latter writes post-deployment retrieval or dependency context. Persistence and remediation differ.	The same payload and victim family, with poison count or rate separated from retrieval opportunities and with persistence measured after record removal or retraining.
Asleep at the Keyboard [112] vs. INSEC [65]	One estimates insecure generation under ordinary prompts; the other optimizes a current input to induce insecurity. A prevalence estimate is not an attack success rate.	The same task sample, model version, decoding policy, security oracle, and denominator, with ordinary and adversarial prompts reported separately.
RedCode [42] vs. Your AI, My Shell [91]	RedCode supplies direct harmful requests to code agents and checks their actions in Docker; Your AI, My Shell plants runtime context consumed during a later task. Both involve agent execution, but their entry points and success oracles differ.	A shared agent environment that reports context selection, model interpretation, authorization, execution, and terminal effect under the same tool permissions.

6.5.1 Mechanisms and Native Budgets. Table 22 connects each entry point to the mechanism, failure condition, and required evidence.

6.5.2 Adaptation, Utility, and Reproducibility. Native budgets differ by surface: queries, edits, samples, or tokens for input attacks; poison records, corpus proportion, and compute for training; stored artifacts, retrieval opportunities, and runs for runtime attacks. Table 23 lists the corresponding comparison fields.

Fixed attacks test a known procedure, held-out attacks test new procedures or samples, transfer crosses a specified model or data boundary, and adaptive attacks use knowledge of the defense. Evaluations should retain attacker and defender feedback, tuning budgets, and clean utility under the same policy. Relevant costs include false rejection, task failure, empty outputs, compute, latency, analyzer calls, and human intervention; privacy studies also require accounting assumptions.

Table 22. Mechanism-level comparison across representative attack families.

Family	State changed	Optimization lever	Characteristic failure	Evidence needed
Identifier evasion	Current source program	Name candidates, positions, queries, gradients, or scores	Binding, reflection, or API-visible name changes invalidate the preservation premise	Label/rank change plus preservation support
Structural evasion	Current source, AST, graph, or binary	Rewrite library, placement, search, and validity filtering	Hidden side effects or representation-specific invariants	Task metric, paired tests, or equivalence argument
Training backdoor	Training records or weights	Trigger, poison rate, target association, and training schedule	Clean degradation, near-trigger activation, or removal during fine-tuning	Triggered success, clean utility, specificity, and persistence
Retrieval poisoning	Post-deployment knowledge store	Artifact content, indexing, retrieval rank, and persistence	Artifact is not selected or is ignored after selection	Insertion, retrieval, harmful output, and downstream effect
Repository/tool injection	Files, rules, skills, descriptions, or tool output	Instruction placement and available authority	Policy blocks the proposal or the sandbox contains execution	Selection, proposal, authorization, execution, and terminal effect
Output/privacy risk	Model outputs, training relationship, or ecosystem artifacts	Sampling policy, diagnostic probes, or artifact selection	Measurement conflates warnings, functional failures, and exploitable outcomes	Static, dynamic, exploit, provenance, or registry evidence

Model identifiers, splits, prompts, transformations, analyzers, tests, and environments connect each result to an evaluated configuration. Agents additionally require tool permissions and traces. Parsing failures, failed builds, refusals, and unsuccessful attacks remain in the relevant denominator.

7 Threats to Validity

Search coverage. The initial collection's exports and screening counts were not retained. The October supplementary search is logged and applies the June 2026 cutoff, but index coverage and ranking can omit relevant work.

Selection and source availability. Preprints and first-party reports vary in detail and archival status. Excluding unverifiable protocols can introduce source-availability bias.

Table 23. Comparison fields for each attack surface, risk, or defense.

Boundary	Attacker control and budget	Validity and outcome	Utility and reproducibility
Current inference input	Feedback available to the attacker; number of queries, edits, samples, and tokens	Accepted and rejected transformations; preservation support; model, static, dynamic, and terminal outcomes	Clean task performance, model and decoding settings, original and transformed inputs
Training state	Exact write point; poison count and corpus denominator; filtering, training schedule, and compute	Triggered behavior, near-trigger specificity, clean behavior, payload functionality, and persistence	Clean utility, false removals, poison records, training recipe, and checkpoint identity
Runtime context	Writable artifact, persistence, retrieval opportunities, feedback, and agent runs	Insertion, selection, interpretation, action proposal, authorization, execution, and terminal effect	Benign retrieval and task success, permissions, tool schemas, and traces
Output and ecosystem risk	Sampling source and date, model version, decoding, retries, and registry or repository snapshot	Counting unit and denominator; analyzer rules, tests, exploits, matching threshold, or provenance evidence	Functional pass rate, uncertainty, raw outputs, and environment
Defenses	Fixed, held-out, transfer, and adaptive attacker budgets	Protected boundary and security outcome under the defender's stated assumptions	Clean utility, false rejection, latency, compute, and release artifacts

CodeShield uses its April 2024 deployment report and method documentation. The large GitHub study has inconsistent file totals, so we retain its CWE-occurrence count without deriving file-level prevalence [128]. Black et al.'s artifact has two upload/path security checks that return failure regardless of file state; its other shell/file witnesses support execution evidence, but these checks limit aggregate security-rate interpretation [10].

Coding judgment. Coding was conducted by one author with subsequent model-assisted source checks, without an independent human coder or inter-rater estimate. Preservation contracts, execution purpose, and diagnostic privacy roles involve interpretation. The companion data retain locators and decisions for author review.

Measurement and generalization. Benchmark tests, local symbolic checks, and sandbox effects cover different behavior. Sampling policies, analyzers, environments, and denominators limit pooling. Changes to hosted models, dependencies, and source versions can alter later results.

8 Challenges and Opportunities

8.1 Challenges

8.1.1 Preserving the Relevant Program Behavior. Transformation-based evaluation requires a task-relevant contract covering names, side effects, exceptions, dependencies, or binary

state. Construction rules, benchmark tests, and symbolic checks cover different parts of that contract.

8.1.2 Maintaining Trust across an Agent's Runtime Path. Agent controls act at different stages, from context selection to execution. Their composition needs evaluation that identifies where each control interrupts an attack and which later effects remain reachable.

8.1.3 Comparing Adaptive Protection and Benign Utility. Transformation, trigger, and analyzer specialization can leave held-out attacks effective. Adaptive protection must be assessed alongside benign functionality, false rejection, and operating cost. Privacy probes and watermark-removal attacks face the same need for held-out access paths.

8.1.4 Retaining Evidence as Systems Change. Updates to models, retrieval, registries, tools, and dependencies change functionality and exploitability. Longitudinal evaluation must separate system drift from changes in the measurement procedure.

8.2 Opportunities

8.2.1 Executable Preservation Collections. Quiring et al., CodeCrash, CoTR, PromSec, and FunPoison provide paired-check examples [76, 103, 121, 165, 171]. A shared collection could retain original/transformed pairs, preconditions, rejected variants, builds, inputs, and observed outputs, enabling later studies to extend a stated contract.

8.2.2 Shared Traces for Coding-Agent Controls. Replaying the same retrieval, tool, and skill attacks under provenance marking, context restrictions, control/data separation, capability scoping, and sandbox policies would isolate each control's contribution. Stage-level outcomes, benign task success, and intervention cost would support comparisons across layers.

8.2.3 Matched Transfer and Adaptive Evaluation. Separate tracks could test transfer across seeds, checkpoints, architectures, datasets, tasks, and languages. Development and held-out attacks, both sides' feedback, tuning budgets, utility, and false rejection would expose the cost and coverage of adaptation.

8.2.4 Longitudinal and Versioned Benchmarks. Dated models, environments, prompts, analyzer rules, and raw outcomes would support reruns after updates. Traces could locate changes in retrieval, generation, authorization, execution, or the oracle, making drift a measured property.

9 Conclusion

We reviewed 148 works and 219 contributions across neural code attacks, risks, defenses, and evaluation. Organizing attacks by entry into training state, current input, or runtime context connects model failures to the software systems that expose them. Preservation contracts and outcome oracles explain what each reported result establishes. Within-study comparisons show family-specific protection and security–utility trade-offs. Executable preservation collections, shared agent traces, adaptive tests, and versioned benchmarks offer concrete ways to improve this evidence.

Ethics statement. We analyze publicly disclosed studies and report credential findings in aggregate without reproducing secrets.

References

[1] Vibhor Agarwal, Yulong Pei, Salwa Alamir, and Xiaomo Liu. 2024. CodeMirage: Hallucinations in Code Generated by Large Language Models. arXiv preprint arXiv:2408.08333. arXiv:2408.08333 https://arxiv.org/html/2408.08333v1

[2] Hojjat Aghakhani, Wei Dai, Andre Manoel, Xavier Fernandes, Anant Kharkar, Christopher Kruegel, Giovanni Vigna, David Evans, Benjamin Zorn, and Robert Sim. 2024. TrojanPuzzle: Covertly Poisoning Code-Suggestion Models. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, Piscataway, NJ, USA, 1122–1140. doi:10.1109/sp54263.2024.00140

[3] Ali Al-Kaswan, Maliheh Izadi, and Arie van Deursen. 2024. Traces of Memorisation in Large Language Models for Code. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3597503.3639133

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv preprint arXiv:2108.07732. https://arxiv.org/abs/2108.07732 arXiv preprint.

[5] MD Abdul Awal, Mrigank Rochan, and Chanchal K. Roy. 2025. Investigating Adversarial Attacks in Software Analytics via Machine Learning Explainability. *Software Quality Journal* 33, 3 (2025), 31. doi:10.1007/s11219-025-09727-2

[6] Enna Basic and Alberto Giaretta. 2026. From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security. arXiv preprint arXiv:2412.15004. arXiv:2412.15004 [cs.CR] doi:10.48550/arXiv.2412.15004 Version 4, 17 March 2026.

[7] Manish Bhatt, Sahana Chennabasappa, Yue Li, Cyrus Nikolaidis, Daniel Song, Shengye Wan, Faizan Ahmad, Cornelius Aschermann, Yaohui Chen, Dhaval Kapil, David Molnar, Spencer Whitman, and Joshua Saxe. 2024. CyberSecEval 2: A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models. arXiv preprint arXiv:2404.13161. https://arxiv.org/abs/2404.13161 arXiv preprint.

[8] Manish Bhatt, Sahana Chennabasappa, Cyrus Nikolaidis, Shengye Wan, Ivan Evtimov, Dominik Gabi, Daniel Song, Faizan Ahmad, Cornelius Aschermann, Lorenzo Fontana, Sasha Frolov, Ravi Prakash Giri, Dhaval Kapil, Yiannis Kozyrakis, David LeBlanc, James Milazzo, Aleksandar Straumann, Gabriel Synnaeve, Varun Vontimitta, Spencer Whitman, and Joshua Saxe. 2023. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. arXiv preprint arXiv:2312.04724. https://arxiv.org/abs/2312.04724 arXiv preprint.

[9] Pavol Bielik and Martin Vechev. 2020. Adversarial Robustness for Code. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, Brookline, MA, USA, 896–907. https://proceedings.mlr.press/v119/bielik20a.html

[10] Gavin Black, Bhaskar Prasad Rimal, and Varghese Mathew Vaidyan. 2025. Balancing Security and Correctness in Code Generation: An Empirical Study on Commercial Large Language Models. *IEEE Transactions on Emerging Topics in Computational Intelligence* 9, 1 (2025), 419–430. doi:10.1109/TETCI.2024.3446695

[11] Aylin Caliskan-Islam, Richard Harang, Andrew Liu, Arvind Narayanan, Clare Voss, Fabian Yamaguchi, and Rachel Greenstadt. 2015. De-anonymizing Programmers via Code Stylometry. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Berkeley, CA, USA, 255–270. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/caliskan-islam

[12] Gianluca Capozzi, Daniele Cono D'Elia, Giuseppe Antonio Di Luna, and Leonardo Querzoni. 2024. Adversarial Attacks Against Binary Similarity Systems. *IEEE Access* 12 (2024), 161247–161269. doi:10.1109/access.2024.3488204

[13] Nicholas Carlini, Matthew Jagielski, Christopher A. Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. 2024. Poisoning Web-Scale Training Datasets Is Practical. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, Piscataway, NJ, USA, 407–425. doi:10.1109/SP54263.2024.00179

[14] Melih Catal, Pooja Rani, and Harald C. Gall. 2025. Towards Privacy-Preserving Code Generation: Differentially Private Code Language Models. arXiv preprint arXiv:2512.11482. arXiv:2512.11482 https://arxiv.org/html/2512.11482v1

[15] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. doi:10.1109/tse.2021.3087402

[16] Shuyu Chang, Haiping Huang, Yanjun Zhang, Yujin Huang, Fu Xiao, and Leo Yu Zhang. 2026. Transferable Backdoor Attacks for Code Models via Sharpness-Aware Adversarial Perturbation. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 1 (2026), 57–65. doi:10.1609/aaai.v40i1.36964

[17] Gong Chen, Wenjie Liu, Xiaoyuan Xie, Xunzhu Tang, Tegawendé F. Bissyandé, and Songqiang Chen. 2025. MT4DP: Data Poisoning Attack Detection for DL-based Code Search Models via Metamorphic Testing. arXiv preprint arXiv:2507.11092. https://arxiv.org/abs/2507.11092 arXiv preprint.

[18] Junkai Chen, Huihui Huang, Yunbo Lyu, Junwen An, Jieke Shi, Chengran Yang, Ting Zhang, Haoye Tian, Yikun Li, Zhenhao Li, Xin Zhou, Xing Hu, and David Lo. 2025. SecureVibeBench: Benchmarking Secure Vibe Coding of AI Agents via Reconstructing Vulnerability-Introducing Scenarios. arXiv preprint arXiv:2509.22097. arXiv:2509.22097 https://arxiv.org/html/2509.22097v5

[19] Jiachi Chen, Qingyuan Zhong, Yanlin Wang, Kaiwen Ning, Yongkun Liu, Zenan Xu, Zhe Zhao, Ting Chen, and Zibin Zheng. 2024. RMCBench: Benchmarking Large Language Models' Resistance to Malicious Code. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 995–1006. doi:10.1145/3691620.3695480

[20] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374. https://arxiv.org/abs/2107.03374 arXiv preprint.

[21] Mingjie Chen, Tiancheng Zhu, Mingxue Zhang, Yiling He, Minghao Lin, Penghui Li, and Kui Ren. 2025. Explainer-guided Targeted Adversarial Attacks Against Binary Code Similarity Detection Models. arXiv preprint arXiv:2506.05430. https://arxiv.org/abs/2506.05430 arXiv preprint.

[22] Nuo Chen, Qiushi Sun, Jianing Wang, Ming Gao, Xiaoli Li, and Xiang Li. 2023. Evaluating and Enhancing the Robustness of Code Pre-trained Models through Structure-Aware Adversarial Samples Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Stroudsburg, PA, USA, 14857–14873. doi:10.18653/v1/2023.findings-emnlp.991

[23] Siran Chen, Jingzheng Wu, Xiang Ling, Tianyue Luo, Jiayu Liu, and Yanjun Wu. 2024. Reinforcement-Learning-based Adversarial Attacks Against Vulnerability Detection Models. *Journal of Software* 35, 8 (2024), 3647–3667. doi:10.13328/j.cnki.jos.007120 In Chinese; translated title. Selected primary protocol; related English version: DOI 10.21655/ijsi.1673-7288.00332.

[24] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. 2023. DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. Association for Computing Machinery, New York, NY, USA, 654–668. doi:10.1145/3607199.3607242

[25] Yuchen Chen, Weisong Sun, Chunrong Fang, Zhenpeng Chen, Yifei Ge, Tingxu Han, Quanjun Zhang, Yang Liu, Zhenyu Chen, and Baowen Xu. 2026. Security of Language Models for Code: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 35, 7 (2026), 1–65. doi:10.1145/3735554

[26] Zhaoyang Chu, Yao Wan, Zhikun Zhang, Di Wang, Zhou Yang, Hongyu Zhang, Pan Zhou, Xuanhua Shi, Hai Jin, and David Lo. 2026. Scrub It Out! Erasing Sensitive Memorization in Code Language Models via Machine Unlearning. In *48th IEEE/ACM International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 13 pages. arXiv:2509.13755 [cs.SE] doi:10.1145/3744916.3764573

[27] Domenico Cotroneo, Cristina Improta, Pietro Liguori, and Roberto Natella. 2024. Vulnerabilities in AI Code Generators: Exploring Targeted Data Poisoning Attacks. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC)*. Association for Computing Machinery, New York, NY, USA, 280–292. doi:10.1145/3643916.3644416

[28] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv preprint arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[29] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 26 pages. doi:10.52202/079017-2636

[30] Yangruibo Ding, Luca Buratti, Saurabh Pujar, Alessandro Morari, Baishakhi Ray, and Saikat Chakraborty. 2022. Towards Learning (Dis)-Similarity of Source Code from Program Contrasts (DISCO). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 6300–6312. doi:10.18653/v1/2022.acl-long.436

[31] Zeming Dong, Qiang Hu, Yuejun Guo, Maxime Cordy, Mike Papadakis, Zhenya Zhang, Yves Le Traon, and Jianjun Zhao. 2023. MixCode: Enhancing Code Classification by Mixup-Based Data Augmentation. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Piscataway, NJ, USA, 379–390. doi:10.1109/saner56733.2023.00043

[32] Zeming Dong, Qiang Hu, Xiaofei Xie, Maxime Cordy, Mike Papadakis, Yves Le Traon, and Jianjun Zhao. 2026. GenCode: A Generic Data Augmentation Framework for Boosting Deep Learning-Based Code Understanding. *Empirical Software Engineering* 31, 3 (2026), 72. doi:10.1007/s10664-026-10809-3

[33] Xiaohu Du, Ming Wen, Zichao Wei, Shangwen Wang, and Hai Jin. 2023. An Extensive Study on Adversarial Attack against Pre-trained Models of Code. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 489–501. doi:10.1145/3611643.3616356

[34] Yanjun Fu, Ethan Baker, Yu Ding, and Yizheng Chen. 2024. Constrained Decoding for Secure Code Generation. arXiv preprint arXiv:2405.00218. https://arxiv.org/abs/2405.00218 arXiv preprint.

[35] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 218:1–218:34. doi:10.1145/3716848

[36] Shuzheng Gao, Eric John Li, Man Ho Lam, Jingyu Xiao, Yuxuan Wan, Chaozheng Wang, Man Tik Ng, and Michael R. Lyu. 2025. TREAT: A Code LLMs Trustworthiness / Reliability Evaluation and Testing Framework. arXiv preprint arXiv:2510.17163. https://arxiv.org/abs/2510.17163 arXiv preprint.

[37] Hongliang Ge, Wenkang Zhong, Chuanyi Li, Jidong Ge, Hao Hu, and Bin Luo. 2024. RobustNPR: Evaluating the Robustness of Neural Program Repair Models. *Journal of Software: Evolution and Process* 36, 4 (2024), e2586. doi:10.1002/smr.2586

[38] Ehab Ghannoum and Mohammad Ghafari. 2025. Poisoned Source Code Detection in Code Models. *Journal of Systems and Software* 226 (2025), 112384. doi:10.1016/j.jss.2025.112384

[39] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985

[40] Shanzhi Gu, Zhaoyang Qu, Ruotong Geng, Mingyang Geng, Shangwen Wang, Chuanfu Xu, Haotian Wang, Zhipeng Lin, and Dezun Dong. 2026. Mitigating Sensitive Information Leakage in LLMs4Code through Machine Unlearning. *Neural Networks* 198 (2026), 108606. doi:10.1016/j.neunet.2026.108606

[41] Batu Guan, Yao Wan, Zhangqian Bi, Zheng Wang, Hongyu Zhang, Pan Zhou, and Lichao Sun. 2024. CodeIP: A Grammar-Guided Multi-Bit Watermark for Large Language Models of Code. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Stroudsburg, PA, USA, 9243–9258. doi:10.18653/v1/2024.findings-emnlp.541

[42] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. 2024. RedCode: Risky Code Execution and Generation Benchmark for Code Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. Curran Associates, Inc., Red Hook, NY, USA, 106190–106236. doi:10.52202/079017-3369

[43] Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. 2025. CodeMirage: A Multi-Lingual Benchmark for Detecting AI-Generated and Paraphrased Source Code from Production-Level LLMs. arXiv preprint arXiv:2506.11059. https://arxiv.org/abs/2506.11059

[44] Hossein Hajipour, Keno Hassler, Thorsten Holz, Lea Schönherr, and Mario Fritz. 2024. CodeLMSec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, Piscataway, NJ, USA, 684–709. doi:10.1109/satml59370.2024.00040

[45] Sivana Hamer, Marcelo d'Amorim, and Laurie Williams. 2024. Just Another Copy and Paste? Comparing the Security Vulnerabilities of ChatGPT Generated Code and StackOverflow Answers. In *2024 IEEE Security and Privacy Workshops (SPW)*. IEEE, Piscataway, NJ, USA, 87–94. doi:10.1109/SPW63631.2024.00014

[46] Tingxu Han, Wei Song, Weisong Sun, Hao Wu, Chunrong Fang, Yuan Xiao, Xiaofang Zhang, Zhenyu Chen, and Yang Liu. 2026. Enhancing and Reporting Robustness Boundary of Neural Code Models for Intelligent Code Understanding. arXiv preprint arXiv:2603.24119. https://arxiv.org/abs/2603.24119

[47] Jingxuan He and Martin Vechev. 2023. Large Language Models for Code: Security Hardening and Adversarial Testing (SVEN). In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 1865–1879. doi:10.1145/3576915.3623175

[48] Jingxuan He, Mark Vero, Gabriela Krasnopolska, and Martin Vechev. 2024. Instruction Tuning for Secure Code Generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, Brookline, MA, USA, 18043–18062. https://proceedings.mlr.press/v235/he24k.html

[49] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1. Curran Associates, Inc., Red Hook, NY, USA, 11 pages. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/c24cd76e1ce41366a4bbe8a49b02a028-Abstract-round2.html

[50] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas Reps. 2022. Semantic Robustness of Models of Source Code. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Piscataway, NJ, USA, 526–537. doi:10.1109/saner53432.2022.00070

[51] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv preprint arXiv:2403.14720. https://arxiv.org/abs/2403.14720

[52] Md Imran Hossen, Sai Venkatesh Chilukoti, Liqun Shan, Sheng Chen, Yinzhi Cao, and Xiali Hei. 2024. Double Backdoored: Converting Code Large Language Model Backdoors to Traditional Malware via Adversarial Instruction Tuning Attacks. arXiv preprint arXiv:2404.18567. https://arxiv.org/abs/2404.18567 arXiv preprint.

[53] Md Imran Hossen and Xiali Hei. 2024. On the Adversarial Robustness of Instruction-Tuned Large Language Models for Code. arXiv preprint arXiv:2411.19508. https://arxiv.org/abs/2411.19508 arXiv preprint.

[54] Li Huang, Weifeng Sun, and Meng Yan. 2025. Iterative Generation of Adversarial Example for Deep Code Models. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 2213–2224. doi:10.1109/ICSE55347.2025.00086

[55] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. 2024. Your Code Secret Belongs to Me: Neural Code Completion Tools Can Memorize Hard-Coded Credentials. *Proceedings of the ACM on Software Engineering (FSE)* 1, FSE (2024), 2515–2537. doi:10.1145/3660818

[56] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, et al. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv preprint arXiv:2401.05566. https://arxiv.org/abs/2401.05566

[57] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv preprint arXiv:1909.09436. https://arxiv.org/abs/1909.09436 arXiv preprint.

[58] Aftab Hussain, Md Rafiqul Islam Rabin, Toufique Ahmed, Mohammad Amin Alipour, Bowen Xu, and Stephen Huang. 2025. Finding Trojan Triggers in Code LLMs: An Occlusion-Based Human-in-the-Loop Approach. In *IEEE/ACM International Conference on AI Engineering (CAIN)*. IEEE, Piscataway, NJ, USA, 281–282. doi:10.1109/cain66642.2025.00050

[59] Aftab Hussain, Md Rafiqul Islam Rabin, and Mohammad Amin Alipour. 2023. TrojanedCM: A Repository of Trojaned Large Language Models of Code. arXiv preprint arXiv:2311.14850. https://arxiv.org/abs/2311.14850 arXiv preprint.

[60] Aftab Hussain, Md Rafiqul Islam Rabin, and Mohammad Amin Alipour. 2024. On Trojan Signatures in Large Language Models of Code. arXiv preprint arXiv:2402.16896. https://arxiv.org/abs/2402.16896 arXiv preprint, ICLR 2024 Workshop.

[61] Cristina Improta. 2023. Poisoning Programs by Un-Repairing Code: Security Concerns of AI-Generated Code. In *2023 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, Piscataway, NJ, USA, 128–131. doi:10.1109/ISSREW60843.2023.00060

[62] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping Language to Code in Programmatic Context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 1643–1652. doi:10.18653/v1/d18-1192

[63] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph Gonzalez, and Ion Stoica. 2021. Contrastive Code Representation Learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 5954–5971. doi:10.18653/v1/2021.emnlp-main.482

[64] Mahesh Jamdade and Yi Liu. 2024. A Pilot Study on Secure Code Generation with ChatGPT for Web Applications. In *Proceedings of the 2024 ACM Southeast Conference*. Association for Computing Machinery, New York, NY, USA, 229–234. doi:10.1145/3603287.3651194

[65] Slobodan Jenko, Niels Mündler, Jingxuan He, Mark Vero, and Martin Vechev. 2025. Black-Box Adversarial Attacks on LLM-Based Code Completion. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Brookline, MA, USA, 27050–27069. https://proceedings.mlr.press/v267/jenko25a.html

[66] Akshita Jha and Chandan K. Reddy. 2023. CodeAttack: Code-Based Adversarial Attacks for Pre-trained Programming Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 12 (2023), 14892–14900. doi:10.1609/aaai.v37i12.26739

[67] Jinghan Jia, Shashank Srikant, Tamara Mitrovska, Chuang Gan, Shiyu Chang, Sijia Liu, and Una-May O'Reilly. 2023. CLAWSAT: Towards Both Robust and Accurate Code Models. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Piscataway, NJ, USA, 212–223. doi:10.1109/saner56733.2023.00029

[68] Lichen Jia, Bowen Tang, Chenggang Wu, Zhe Wang, Zihan Jiang, Yuanming Lai, Yan Kang, Ning Liu, and Jingfeng Zhang. 2022. FuncFooler: A Practical Black-Box Attack Against Learning-Based Binary Code Similarity Detection Methods. arXiv preprint arXiv:2208.14191. https://arxiv.org/abs/2208.14191 arXiv preprint.

[69] Xiaojun Jia, Jie Liao, Simeng Qin, Jindong Gu, Wenqi Ren, Xiaochun Cao, Yang Liu, and Philip H. S. Torr. 2026. SkillJect: Effectively Automating Skill-Based Prompt Injection for Skill-Enabled Agents. arXiv preprint arXiv:2602.14211. arXiv:2602.14211 [cs.CR] doi:10.48550/arXiv.2602.14211 Version 3, 16 June 2026.

[70] Xue Jiang, Yihong Dong, Zheng Fang, Yingwei Ma, Tangxinyu Wang, Rongyu Cao, Binhua Li, Zhi Jin, Wenpin Jiao, Yongbin Li, and Ge Li. 2025. Large Language Model Unlearning for Source Code. arXiv preprint arXiv:2506.17125. arXiv:2506.17125 https://arxiv.org/html/2506.17125v2

[71] Yuan Jiang, Zehao Li, Shan Huang, Christoph Treude, Xiaohong Su, and Tiantian Wang. 2025. Effective Code Membership Inference for Code Completion Models via Adversarial Prompts. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Piscataway, NJ, USA, 2312–2324. doi:10.1109/ASE63991.2025.00191

[72] Naizhu Jin, Zhong Li, Yinggang Guo, Chao Su, Tian Zhang, and Qingkai Zeng. 2024. SABER: Model-agnostic Backdoor Attack on Chain-of-Thought in Neural Code Generation. arXiv preprint arXiv:2412.05829. https://arxiv.org/abs/2412.05829 arXiv preprint.

[73] Jonathan Katzy, Răzvan Popescu, Arie van Deursen, and Maliheh Izadi. 2024. An Exploratory Investigation into Code License Infringements in Large Language Model Training Datasets. In *Proceedings of the 1st IEEE/ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*. Association for Computing Machinery, New York, NY, USA, 74–85.

doi:10.1145/3650105.3652298

[74] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure is Code Generated by ChatGPT?. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, Piscataway, NJ, USA, 2445–2451. doi:10.1109/smc53992.2023.10394237

[75] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Brookline, MA, USA, 18319–18345. https://proceedings.mlr.press/v202/lai23b.html

[76] Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM Fragility to Misleading Natural Language in Code Reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., Red Hook, NY, USA, 133886–133932. doi:10.52202/085713-4029

[77] Quoc Hung Le, Thanh Le-Cong, Bach Le, and Bowen Xu. 2025. Signature in Code Backdoor Detection, how far are we? arXiv preprint arXiv:2510.13992. arXiv:2510.13992 https://arxiv.org/html/2510.13992v1

[78] Thanh Le-Cong, Dat Nguyen, Bach Le, and Toby Murray. 2025. Towards Reliable Evaluation of Neural Program Repair with Natural Robustness Testing. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 34, 7, Article 213 (2025), 44 pages. doi:10.1145/3716167

[79] Taehyun Lee, Seokhee Hong, Jaewoo Ahn, Ilgee Hong, Hwaran Lee, Sangdoo Yun, Jamin Shin, and Gunhee Kim. 2024. Who Wrote this Code? Watermarking for Code Generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 4890–4911. doi:10.18653/v1/2024.acl-long.268

[80] Boquan Li, Zirui Fu, Mengdi Zhang, Peixin Zhang, Jun Sun, and Xingmei Wang. 2025. Efficient and Universal Watermarking for LLM-Generated Code Detection. arXiv preprint arXiv:2402.07518v4. https://arxiv.org/abs/2402.07518v4 Version 4, 1 August 2025.

[81] Haoyang Li, Huan Gao, Zhiyuan Zhao, Zhiyu Lin, Junyu Gao, and Xuelong Li. 2025. LLMs Caught in the Crossfire: Malware Requests and Jailbreak Challenges. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 27833–27848. doi:10.18653/v1/2025.acl-long.1350

[82] Jia Li, Zhuo Li, Huangzhao Zhang, Ge Li, Zhi Jin, Xing Hu, and Xin Xia. 2024. Poison Attack and Poison Detection on Deep Source Code Processing Models. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 33, 3, Article 62 (2024), 31 pages. doi:10.1145/3630008

[83] Xueyang Li, Guozhu Meng, Shangqing Liu, Lu Xiang, Kun Sun, Kai Chen, Xiapu Luo, and Yang Liu. 2024. Attribution-guided Adversarial Code Prompt Generation for Code Completion Models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 1460–1471. doi:10.1145/3691620.3695517

[84] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158

[85] Yanzhou Li, Shangqing Liu, Kangjie Chen, Xiaofei Xie, Tianwei Zhang, and Yang Liu. 2023. Multi-target Backdoor Attacks for Code Pre-trained Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 7236–7254. doi:10.18653/v1/2023.acl-long.399

[86] Zhen Li, Qian (Guenevere) Chen, Chen Chen, Yayi Zou, and Shouhuai Xu. 2022. RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 1906–1918. doi:10.1145/3510003.3510181

[87] Bo Lin, Shangwen Wang, Liqian Chen, and Xiaoguang Mao. 2026. Exploring the Security Threats of Knowledge Base Poisoning in Retrieval-Augmented Code Generation. *IEEE Transactions on Software Engineering* 52, 8 (2026), 2250–2267. doi:10.1109/tse.2026.3689066

[88] Meiyu Lin, Haichuan Zhang, Jiale Lao, Renyuan Li, Yuanchun Zhou, Carl Yang, Yang Cao, and Mingjie Tang. 2025. Are Your LLM-based Text-to-SQL Models Secure? Exploring SQL Injection via

Backdoor Attacks. *Proceedings of the ACM on Management of Data (PACMMOD)* 3, 6 (2025), 1–27. doi:10.1145/3769762

[89] Shigang Liu, Di Cao, Junae Kim, Tamas Abraham, Paul Montague, Seyit Camtepe, Jun Zhang, and Yang Xiang. 2024. EaTVul: ChatGPT-based Evasion Attack Against Software Vulnerability Detection. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Berkeley, CA, USA, 7357–7374. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-shigang

[90] Shangqing Liu, Bozhi Wu, Xiaofei Xie, Guozhu Meng, and Yang Liu. 2023. ContraBERT: Enhancing Code Pre-trained Models via Contrastive Learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 2476–2487. doi:10.1109/icse48619.2023.00207

[91] Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. 2025. "Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors. arXiv preprint arXiv:2509.22040. https://arxiv.org/abs/2509.22040

[92] Zheng Liu, Chen Gong, Terry Yue Zhuo, Kecen Li, Weichen Yu, Matt Fredrikson, and Tianhao Wang. 2026. PrivCode: When Code Generation Meets Differential Privacy. In *Proceedings of the 2026 Network and Distributed System Security Symposium*. Internet Society, Reston, VA, USA, 20 pages. doi:10.14722/ndss.2026.240936

[93] Zheng Liu, Chen Gong, Terry Yue Zhuo, Zhou Yang, Kecen Li, Wenlong Meng, Xinwen Hou, Yu Liu, and Xiaochen Li. 2026. PrivCode++: Latent-Conditioned Differentially Private Code Generation for Comprehensive Guarantees. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, Brookline, MA, USA, 76332–76353. arXiv:2606.09145 [cs.CR] https://proceedings.mlr.press/v306/liu26ao.html Protocol coding uses the pre-cutoff author version, arXiv:2606.09145v1.

[94] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1. Curran Associates, Inc., Red Hook, NY, USA, 16 pages. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html

[95] Narek Maloyan and Dmitry Namiot. 2026. Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems. *International Journal of Open Information Technologies* 14, 2 (2026), 1–10. arXiv:2601.17548 [cs.CR] doi:10.48550/arXiv.2601.17548

[96] Lakshmi Likhitha Mankali, Jitendra Bhandari, Manaar Alam, Ramesh Karri, Michail Maniatakos, Ozgur Sinanoglu, and Johann Knechtel. 2025. RTL-Breaker: Assessing the Security of LLMs Against Backdoor Attacks on HDL Code Generation. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE, Piscataway, NJ, USA, 1–7. doi:10.23919/DATE64628.2025.10993260

[97] Meta. 2024. Our Responsible Approach to Meta AI and Meta Llama 3. https://ai.meta.com/blog/meta-llama-3-meta-ai-responsibility/. 18 April. First-party CodeShield deployment report; method artifact: https://github.com/meta-llama/PurpleLlama/tree/main/CodeShield.

[98] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional Neural Networks over Tree Structures for Programming Language Processing. *Proceedings of the AAAI Conference on Artificial Intelligence* 30, 1 (2016), 1287–1293. doi:10.1609/aaai.v30i1.10139

[99] Zahra Mousavi, Chadni Islam, Kristen Moore, Alsharif Abuadbba, and M. Ali Babar. 2024. An Investigation into Misuse of Java Security APIs by Large Language Models. In *ACM Asia Conference on Computer and Communications Security (ASIACCS)*. Association for Computing Machinery, New York, NY, USA, 1299–1315. doi:10.1145/3634737.3661134

[100] Fangwen Mu, Junjie Wang, Zhuohao Yu, Lin Shi, Song Wang, Mingyang Li, and Qing Wang. 2024. CodePurify: Defend Backdoor Attacks on Neural Code Models via Entropy-based Purification. arXiv preprint arXiv:2410.20136. https://arxiv.org/abs/2410.20136

[101] Wenhan Mu, Ling Xu, Shuren Pei, Le Mi, and Huichi Zhou. 2025. Evaluate-and-Purify: Fortifying Code Language Models Against Adversarial Attacks Using LLM-as-a-Judge. In *Advanced Intelligent Computing Technology and Applications*. Springer, Cham, Switzerland, 369–379. doi:10.1007/978-981-96-9994-0_31

[102] CheolWon Na, YunSeok Choi, and Jee-Hyong Lee. 2023. DIP: Dead code Insertion based Black-box Attack for Programming Language Model. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 7777–7791. doi:10.18653/v1/2023.acl-long.430

[103] Mahmoud Nazzal, Issa Khalil, Abdallah Khreishah, and NhatHai Phan. 2024. PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 2266–2280. doi:10.1145/3658644.3690298

[104] Claudia Negri-Ribalta, Rémi Geraud-Stewart, Anastasia Sergeeva, and Gabriele Lenzini. 2024. A Systematic Literature Review on the Impact of AI Models on the Security of Code Generation. *Frontiers in Big Data* 7 (2024), 1386720. doi:10.3389/fdata.2024.1386720

[105] Khoa Nguyen, Khiem Ton, NhatHai Phan, Issa Khalil, Khang Tran, Cristian Borcea, Ruoming Jin, Abdallah Khreishah, and My T. Thai. 2026. NOIR: Privacy-Preserving Generation of Code with Open-Source LLMs. arXiv preprint arXiv:2601.16354. arXiv:2601.16354 https://arxiv.org/html/2601.16354v1

[106] Phuong T. Nguyen, Juri Di Rocco, Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Massimiliano Di Penta. 2024. GPTSniffer: A CodeBERT-based Classifier to Detect Source Code Written by ChatGPT. *Journal of Systems and Software* 214 (2024), 112059. doi:10.1016/j.jss.2024.112059

[107] Thanh-Dat Nguyen, Zhou Yang, Xuan Bach D. Le, Patanamon Thongtanunam, and David Lo. 2023. Adversarial Attacks on Code Models with Discriminative Graph Patterns. arXiv preprint arXiv:2308.11161. https://arxiv.org/abs/2308.11161

[108] Yuzhou Nie, Zhun Wang, Yu Yang, Ruizhe Jiang, Yuheng Tang, Xander Davies, Yarin Gal, Bo Li, Wenbo Guo, and Dawn Song. 2025. SeCodePLT: A Unified Benchmark for Evaluating the Security Risks and Capabilities of Code GenAI. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. Curran Associates, Inc., Red Hook, NY, USA, 14835–14877. doi:10.52202/085713-0447

[109] Georgios Nikitopoulos, Konstantina Dritsa, Panos Louridas, and Dimitris Mitropoulos. 2021. CrossVul: A Cross-Language Vulnerability Dataset with Commit Data. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 1565–1569. doi:10.1145/3468264.3473122

[110] Liang Niu, Shujaat Mirza, Zayd Maradni, and Christina Pöpper. 2023. CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Berkeley, CA, USA, 2133–2150. https://www.usenix.org/conference/usenixsecurity23/presentation/niu

[111] Sheng Ouyang, Yihao Qin, Bo Lin, Liqian Chen, Xiaoguang Mao, and Shangwen Wang. 2025. Smoke and Mirrors: Jailbreaking LLM-based Code Generation via Implicit Malicious Prompts. arXiv preprint arXiv:2503.17953. https://arxiv.org/abs/2503.17953 Presented at the ICSE 2026 Research Track.

[112] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, Piscataway, NJ, USA, 754–768. doi:10.1109/sp46214.2022.9833571

[113] Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. 2025. CWEval: Outcome-driven Evaluation on Functionality and Security of LLM Code Generation. In *International Workshop on Large Language Models for Code (LLM4Code) at ICSE*. IEEE, Piscataway, NJ, USA, 33–40. doi:10.1109/llm4code66737.2025.00009

[114] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157

[115] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1. Curran Associates, Inc., Red Hook, NY, USA, 13 pages. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/a5bfc9e07964f8dddeb95fc584cd965d-Abstract-round2.html

[116] Shiyi Qi, Yuanhang Yang, Shuzheng Gao, Cuiyun Gao, and Zenglin Xu. 2023. BadCS: A Backdoor Attack Framework for Code Search. arXiv preprint arXiv:2305.05503. https://arxiv.org/abs/2305.05503 arXiv preprint.

[117] Weiliang Qi, Jiahao Cao, Darsh Poddar, Sophia Li, and Xinda Wang. 2024. Enhancing Pre-trained Language Models for Vulnerability Detection via Semantic-Preserving Data Augmentation. In *International Conference on Security and Privacy in Communication Networks (SecureComm)*. Springer, Cham, Switzerland, 184–203. doi:10.1007/978-3-031-94458-1_9

[118] Yubin Qu, Song Huang, Xiang Chen, Tongtong Bai, and Yongming Yao. 2025. An Input-Denoising-Based Defense Against Stealthy Backdoor Attacks in Large Language Models for Code. *Information and Software Technology* 180 (2025), 107661. doi:10.1016/j.infsof.2024.107661

[119] Yubin Qu, Song Huang, and Peng Nie. 2025. A Review of Backdoor Attacks and Defenses in Code Large Language Models: Implications for Security Measures. *Information and Software Technology* 182 (2025), 107707. doi:10.1016/j.infsof.2025.107707

[120] Yubin Qu, Song Huang, and Yongming Yao. 2024. A Survey on Robustness Attacks for Deep Code Models. *Automated Software Engineering* 31, 2, Article 65 (2024), 40 pages. doi:10.1007/s10515-024-00464-7

[121] Erwin Quiring, Alwin Maier, and Konrad Rieck. 2019. Misleading Authorship Attribution of Source Code using Adversarial Learning. In *28th USENIX Security Symposium*. USENIX Association, Santa Clara, CA, USA, 479–496. https://www.usenix.org/system/files/sec19-quiring.pdf

[122] Goutham Ramakrishnan and Aws Albarghouthi. 2022. Backdoors in Neural Models of Source Code. In *2022 26th International Conference on Pattern Recognition (ICPR)*. IEEE, Piscataway, NJ, USA, 2892–2899. doi:10.1109/icpr56361.2022.9956690

[123] Avilash Rath, Weiliang Qi, Youpeng Li, and Xinda Wang. 2025. NatGVD: Natural Adversarial Example Attack towards Graph-based Vulnerability Detection. In *2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, Piscataway, NJ, USA, 89–100. doi:10.1109/tps-isa67132.2025.00019

[124] Veselin Raychev, Pavol Bielik, and Martin Vechev. 2016. Probabilistic Model for Code with Decision Trees. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. Association for Computing Machinery, New York, NY, USA, 731–747. doi:10.1145/3022671.2984041

[125] Karl Rubel, Maximilian Noppel, and Christian Wressnegger. 2025. Generalized Adversarial Code-Suggestions: Exploiting Contexts of LLM-based Code-Completion. In *ACM Asia Conference on Computer and Communications Security (ASIACCS)*. Association for Computing Machinery, New York, NY, USA, 358–374. doi:10.1145/3708821.3733870

[126] Rebecca L. Russell, Louis Y. Kim, Lei H. Hamilton, Tomo Lazovich, Jacob A. Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc W. McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning (Draper VDISC). In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, Piscataway, NJ, USA, 757–762. doi:10.1109/ICMLA.2018.00120

[127] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Berkeley, CA, USA, 2205–2222. https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval

[128] Maximilian Schreiber and Pascal Tippe. 2025. Security Vulnerabilities in AI-Generated Code: A Large-Scale Analysis of Public GitHub Repositories. In *International Conference on Information and Communications Security (ICICS)*. Springer, Cham, Switzerland, 153–172. doi:10.1007/978-981-95-3537-8_9

[129] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. 2021. You Autocomplete Me: Poisoning Vulnerabilities in Neural Code Completion. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Berkeley, CA, USA, 1559–1575. https://www.usenix.org/conference/usenixsecurity21/presentation/schuster

[130] Muhammad Usman Shahid, Chuadhry Mujeeb Ahmed, and Rajiv Ranjan. 2025. LLM-CSEC: Empirical Evaluation of Security in C/C++ Code Generated by Large Language Models. arXiv preprint arXiv:2511.18966. arXiv:2511.18966 https://arxiv.org/html/2511.18966v1

[131] Xinyu She, Yue Liu, Yanjie Zhao, Yiling He, Li Li, Chakkrit Tantithamthavorn, Zhan Qin, and Haoyu Wang. 2026. Pitfalls in Language Models for Code Intelligence: A Taxonomy and Survey.

ACM Transactions on Software Engineering and Methodology 35, 3, Article 86 (2026), 42 pages. doi:10.1145/3748647

[132] Chihao Shen, Connor Dilgren, Purva Chiniya, Luke Griffith, Yu Ding, and Yizheng Chen. 2025. SecRepoBench: Benchmarking Code Agents for Secure Code Completion in Real-World Repositories. arXiv preprint arXiv:2504.21205. arXiv:2504.21205 [cs.CR] doi:10.48550/arXiv.2504.21205 Version 3, 14 February 2026.

[133] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. 2025. Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers. In *47th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 1628–1639. doi:10.1109/icse55347.2025.00005

[134] Mohammed Latif Siddiq, Joanna Cecilia da Silva Santos, Sajith Devareddy, and Anna Muller. 2024. SALLM: Security Assessment of Generated Code. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. Association for Computing Machinery, New York, NY, USA, 54–65. doi:10.1145/3691621.3694934

[135] Mohammed Latif Siddiq and Joanna C. S. Santos. 2022. SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. In *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security (MSR4P&S)*. Association for Computing Machinery, New York, NY, USA, 29–33. doi:10.1145/3549035.3561184

[136] Joseph Spracklen, Raveen Wijewickrama, A. H. M. Nazmus Sakib, Anindya Maiti, and Bimal Viswanath. 2025. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, Berkeley, CA, USA, 3687–3706. https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen

[137] Jacob M. Springer, Bryn Marie Reinstadler, and Una-May O'Reilly. 2020. STRATA: Simple, Gradient-Free Attacks for Models of Code. arXiv preprint arXiv:2009.13562. https://arxiv.org/abs/2009.13562

[138] Shashank Srikant, Sijia Liu, Tamara Mitrovska, Shiyu Chang, Quanfu Fan, Gaoyuan Zhang, and Una-May O'Reilly. 2021. Generating Adversarial Computer Programs using Optimized Obfuscations. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Amherst, MA, USA, 16 pages. arXiv:2103.11882 [cs.LG] https://openreview.net/forum?id=PH5PH9ZO_4

[139] Weisong Sun, Yuchen Chen, Chunrong Fang, Yebo Feng, Yuan Xiao, An Guo, Quanjun Zhang, Zhenyu Chen, Baowen Xu, and Yang Liu. 2025. Eliminating Backdoors in Neural Code Models for Secure Code Understanding. *Proceedings of the ACM on Software Engineering (FSE)* 2, FSE (2025), 1386–1408. doi:10.1145/3715782

[140] Weisong Sun, Yuchen Chen, Guanhong Tao, Chunrong Fang, Xiangyu Zhang, Quanjun Zhang, and Bin Luo. 2023. Backdooring Neural Code Search. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 9692–9708. doi:10.18653/v1/2023.acl-long.540

[141] Weisong Sun, Yuchen Chen, Mengzhe Yuan, Chunrong Fang, Zhenpeng Chen, Chong Wang, Yang Liu, Baowen Xu, and Zhenyu Chen. 2025. Show Me Your Code! Kill Code Poisoning: A Lightweight Method Based on Code Naturalness. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, USA, 2663–2675. doi:10.1109/icse55347.2025.00247

[142] Zhensu Sun, Xiaoning Du, Xiapu Luo, Fu Song, David Lo, and Li Li. 2024. FDI: Attack Neural Code Generation Systems through User Feedback Channel. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. Association for Computing Machinery, New York, NY, USA, 528–540. doi:10.1145/3650212.3680300

[143] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. 2023. CodeMark: Imperceptible Watermarking for Code Datasets Against Neural Code Completion Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 1561–1572. doi:10.1145/3611643.3616297

[144] Zhensu Sun, Xiaoning Du, Fu Song, Mingze Ni, and Li Li. 2022. CoProtector: Protect Open-Source Code Against Unauthorized Training Usage with Data Poisoning. In *Proceedings of the ACM Web Conference (WWW)*. Association for Computing Machinery, New York, NY, USA, 652–660. doi:10.1145/3485447.3512225

[145] Jeffrey Svajlenko and Chanchal K. Roy. 2015. Evaluating Clone Detection Tools with BigCloneBench. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE,

Piscataway, NJ, USA, 131–140. doi:10.1109/icsm.2015.7332459

[146] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in Large Language Models Generated Code: An Empirical Study. *Empirical Software Engineering* 30, 3 (2025), 65. doi:10.1007/s10664-025-10614-4

[147] Zhao Tian, Junjie Chen, and Zhi Jin. 2023. Code Difference Guided Adversarial Example Generation for Deep Code Models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Piscataway, NJ, USA, 850–862. doi:10.1109/ase56229.2023.00149

[148] Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhiff, and Riccardo Scandariato. 2024. Prompting Techniques for Secure Code Generation: A Systematic Investigation. arXiv preprint arXiv:2407.07064. arXiv:2407.07064 https://arxiv.org/html/2407.07064v2

[149] Catherine Tony, Markus Mutas, Nicolás E. Díaz Ferreyra, and Riccardo Scandariato. 2023. LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, Piscataway, NJ, USA, 588–592. doi:10.1109/msr59073.2023.00084

[150] Chi-Chien Tsai, Chia-Mu Yu, Ying-Dar Lin, Yu-Sung Wu, and Wei-Bin Lee. 2025. Beyond Natural Language Perplexity: Detecting Dead Code Poisoning in Code Generation Datasets. arXiv preprint arXiv:2502.20246. https://arxiv.org/abs/2502.20246 arXiv preprint.

[151] Maryam Vahdat Pour, Zhuo Li, Lei Ma, and Hadi Hemmati. 2021. A Search-Based Testing Framework for Deep Neural Networks of Source Code Embedding. In *2021 IEEE 14th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Piscataway, NJ, USA, 36–46. doi:10.1109/icst49551.2021.00016

[152] Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, and Martin Vechev. 2025. BaxBench: Can LLMs Generate Correct and Secure Backends?. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, Brookline, MA, USA, 61344–61390. https://proceedings.mlr.press/v267/vero25a.html

[153] Shengye Wan, Cyrus Nikolaidis, Daniel Song, David Molnar, James Crnkovich, Jayson Grace, Manish Bhatt, Sahana Chennabasappa, Spencer Whitman, Stephanie Ding, Vlad Ionescu, Yue Li, and Joshua Saxe. 2024. CyberSecEval 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models. arXiv preprint arXiv:2408.01605. https://arxiv.org/abs/2408.01605 arXiv preprint.

[154] Yao Wan, Guanghua Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Pan Zhou, Hai Jin, and Lichao Sun. 2024. Does Your Neural Code Completion Model Use My Code? A Membership Inference Approach. arXiv preprint arXiv:2404.14296. arXiv:2404.14296 https://arxiv.org/html/2404.14296v2

[155] Yao Wan, Shijie Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Dezhong Yao, Hai Jin, and Lichao Sun. 2022. You See What I Want You to See: Poisoning Vulnerabilities in Neural Code Search. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 1233–1245. doi:10.1145/3540250.3549153

[156] Chenyu Wang, Zhou Yang, Yaniv Harel, and David Lo. 2025. Backdoors in Code Summarizers: How Bad Is It?. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Piscataway, NJ, USA, 1182–1194. doi:10.1109/ase63991.2025.00102

[157] Jiexin Wang, Xitong Luo, Liuwen Cao, Hongkui He, Hailin Huang, Jiayuan Xie, Adam Jatowt, and Yi Cai. 2024. Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval. arXiv preprint arXiv:2407.02395. https://arxiv.org/abs/2407.02395 arXiv preprint.

[158] Shiqi Wang, Zheng Li, Haifeng Qian, Chenghao Yang, Zijian Wang, Mingyue Shang, Varun Kumar, Samson Tan, Baishakhi Ray, Parminder Bhatia, Ramesh Nallapati, Murali Krishna Ramanathan, Dan Roth, and Bing Xiang. 2023. ReCode: Robustness Evaluation of Code Generation Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 13818–13843. doi:10.18653/v1/2023.acl-long.773

[159] Yanlin Wang, Jiadong Wu, Tianyue Jiang, Mingwei Liu, Jiachi Chen, Chong Wang, Ensheng Shi, Xilin Liu, Yuchi Ma, and Zibin Zheng. 2025. DrainCode: Stealthy Energy Consumption Attacks on Retrieval-Augmented Code Generation via Context Poisoning. In *40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Piscataway, NJ, USA, 778–790. doi:10.1109/ase63991.2025.00070

[160] Yanlin Wang, Ziyao Zhang, Chong Wang, Xinyi Xu, Mingwei Liu, Yong Wang, Jiachi Chen, and Zibin Zheng. 2026. RealSec-bench: A Benchmark for Evaluating Secure Code Generation in Real-World Repositories. arXiv preprint arXiv:2601.22706. arXiv:2601.22706 https://arxiv.org/html/2601.22706v1

[161] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2026. MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 42 (2026), 35811–35819. doi:10.1609/aaai.v40i42.40895

[162] Moshi Wei, Yuchao Huang, Jinqiu Yang, Junjie Wang, and Song Wang. 2023. CoCoFuzzing: Testing Neural Code Models with Coverage-Guided Fuzzing. *IEEE Transactions on Reliability* 72, 3 (2023), 1276–1289. doi:10.1109/tr.2022.3208239

[163] Maximilian Wendlinger, Daniel Kowatsch, Konstantin Böttinger, and Philip Sperl. 2026. Security-by-Design for LLM-Based Code Generation: Leveraging Internal Representations for Concept-Driven Steering Mechanisms. arXiv preprint arXiv:2603.11212. arXiv:2603.11212 https://arxiv.org/html/2603.11212v1

[164] Fangzhou Wu, Xiaogeng Liu, and Chaowei Xiao. 2023. DeceptPrompt: Exploiting LLM-driven Code Generation via Adversarial Natural Language Instructions. arXiv preprint arXiv:2312.04730. https://arxiv.org/abs/2312.04730 arXiv preprint.

[165] Yuan Xiao, Jiaming Wang, Yuchen Chen, Wei Song, Jun Sun, Shiqing Ma, Yanzhou Mu, Juan Zhai, Chunrong Fang, Jin Song Dong, and Zhenyu Chen. 2026. Train in Vain: Functionality-Preserving Poisoning to Prevent Unauthorized Use of Code Datasets. arXiv preprint arXiv:2604.22291. arXiv:2604.22291 https://arxiv.org/html/2604.22291v1

[166] Yuchong Xie, Mingyu Luo, Zesen Liu, Zhixiang Zhang, Kaikai Zhang, Yu Liu, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. 2025. Red-Teaming Coding Agents from a Tool-Invocation Perspective: An Empirical Security Assessment. arXiv preprint arXiv:2509.05755. https://arxiv.org/abs/2509.05755

[167] Xiaoyun Xu, Lichao Wu, Jona te Lintelo, Siyu Zhang, and Stjepan Picek. 2026. SPARK: Security Knowledge Priming and Representation-guided Knowledge Activation for LLM-based Secure Code Generation. arXiv preprint arXiv:2606.16244. arXiv:2606.16244 https://arxiv.org/html/2606.16244v1

[168] Hao Yan, Swapneel Suhas Vaidya, Xiaokuan Zhang, and Ziyu Yao. 2025. Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation. arXiv preprint arXiv:2506.23034. https://arxiv.org/abs/2506.23034 arXiv preprint.

[169] Shenao Yan, Shimaa Ahmed, Shan Jin, Sunpreet S. Arora, Yiwei Cai, Yizhen Wang, and Yuan Hong. 2026. Detecting Data Poisoning in Code Generation LLMs via Black-Box, Vulnerability-Oriented Scanning. arXiv preprint arXiv:2603.17174. arXiv:2603.17174 https://arxiv.org/html/2603.17174v1

[170] Shenao Yan, Shen Wang, Yue Duan, Hanbin Hong, Kiho Lee, Doowon Kim, and Yuan Hong. 2024. An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities against Strong Detection. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Berkeley, CA, USA, 1795–1812. https://www.usenix.org/conference/usenixsecurity24/presentation/yan

[171] Guang Yang, Yu Zhou, Xiangyu Zhang, Xiang Chen, Tingting Han, and Taolue Chen. 2025. Assessing and Improving Syntactic Adversarial Robustness of Pre-trained Models for Code Translation. *Information and Software Technology* 181 (2025), 107699. doi:10.1016/j.infsof.2025.107699

[172] Yulong Yang, Haoran Fan, Chenhao Lin, Qian Li, Zhengyu Zhao, Chao Shen, and Xiaohong Guan. 2024. A Survey on Adversarial Machine Learning for Code Data: Realistic Threats, Countermeasures, and Interpretations. arXiv preprint arXiv:2411.07597. https://arxiv.org/abs/2411.07597

[173] Yuchen Yang, Yiming Li, Hongwei Yao, Bingrun Yang, Yiling He, Tianwei Zhang, Dacheng Tao, and Zhan Qin. 2026. ShadowCode: Towards (Automatic) External Prompt Injection Attack against Code LLMs. *IEEE Transactions on Dependable and Secure Computing* (2026), 1–13. doi:10.1109/tdsc.2026.3703498 Early access.

[174] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural Attack for Pre-trained Models of Code. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 1482–1493. doi:10.1145/3510003.3510146

[175] Zhou Yang, Zhensu Sun, Terry Yue Zhuo, Premkumar Devanbu, and David Lo. 2024. Robustness, Security, Privacy, Explainability, Efficiency, and Usability of Large Language Models for Code. arXiv preprint arXiv:2403.07506. https://arxiv.org/abs/2403.07506 arXiv preprint.

[176] Zhou Yang, Bowen Xu, Jie M. Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy Backdoor Attack for Code Models. *IEEE Transactions on Software Engineering* 50, 4 (2024), 721–741. doi:10.1109/tse.2024.3361661

[177] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, DongGyun Han, and David Lo. 2024. Gotcha! This Model Uses My Code! Evaluating Membership Leakage Risks in Code Models. *IEEE Transactions on Software Engineering* 50, 12 (2024), 3290–3306. doi:10.1109/tse.2024.3482719

[178] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, Donggyun Han, and David Lo. 2024. Unveiling Memorization in Code Models. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3597503.3639074

[179] Junyao Ye, Zhen Li, Xi Tang, Shouhuai Xu, Deqing Zou, and Zhongsheng Yuan. 2025. Semantically-Equivalent Transformations-Based Backdoor Attacks against Neural Code Models: Characterization and Mitigation. arXiv:2512.19215 [cs.SE] doi:10.48550/arXiv.2512.19215 Version 1, 22 December 2025.

[180] Tong Ye, Yangkai Du, Tengfei Ma, Lingfei Wu, Xuhong Zhang, Shouling Ji, and Wenhai Wang. 2025. Uncovering LLM-Generated Code: A Zero-Shot Synthetic Code Detector via Code Rewriting. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 1 (2025), 968–976. doi:10.1609/aaai.v39i1.32082

[181] Noam Yefet, Uri Alon, and Eran Yahav. 2020. Adversarial Examples for Models of Code. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30. doi:10.1145/3428230

[182] Chi Zhang, Zifan Wang, Ruoshi Zhao, Ravi Mangal, Matt Fredrikson, Limin Jia, and Corina S. Pasareanu. 2024. Attacks and Defenses for Large Language Models on Coding Tasks. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE, NIER Track)*. Association for Computing Machinery, New York, NY, USA, 2268–2272. doi:10.1145/3691620.3695297

[183] Huangzhao Zhang, Zhiyi Fu, Ge Li, Lei Ma, Zhehao Zhao, Hua'an Yang, Yizhe Sun, Yang Liu, and Zhi Jin. 2022. Towards Robustness of Deep Program Processing Models—Detection, Estimation, and Enhancement. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 3, Article 50 (2022), 40 pages. doi:10.1145/3511887

[184] Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, Yang Liu, and Zhi Jin. 2020. Generating Adversarial Examples for Holding Robustness of Source Code Processing Models. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 01 (2020), 1169–1176. doi:10.1609/aaai.v34i01.5469

[185] Huangzhao Zhang, Shuai Lu, Zhuo Li, Zhi Jin, Lei Ma, Yang Liu, and Ge Li. 2024. CodeBERT-Attack: Adversarial Attack Against Source Code Deep Learning Models via Pre-trained Model. *Journal of Software: Evolution and Process* 36, 3 (2024), e2571. doi:10.1002/smr.2571

[186] Jiale Zhang, Haoxuan Li, Di Wu, Xiaobing Sun, Qinghua Lu, and Guodong Long. 2024. Beyond Dataset Watermarking: Model-Level Copyright Protection for Code Summarization Models. arXiv preprint arXiv:2410.14102. arXiv:2410.14102 https://arxiv.org/html/2410.14102v1

[187] Jie Zhang, Wei Ma, Qiang Hu, Shangqing Liu, Xiaofei Xie, Yves Le Traon, and Yang Liu. 2023. A Black-Box Attack on Code Models via Representation Nearest Neighbor Search. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Stroudsburg, PA, USA, 9706–9716. doi:10.18653/v1/2023.findings-emnlp.649

[188] Sheng Zhang, Hui Li, and Rongrong Ji. 2024. Code Membership Inference for Detecting Unauthorized Data Use in Code Pre-trained Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Stroudsburg, PA, USA, 10593–10603. doi:10.18653/v1/2024.findings-emnlp.621

[189] Yuhao Zhang, Shiqi Wang, Haifeng Qian, Zijian Wang, Mingyue Shang, Linbo Liu, Sanjay Krishna Gouda, Baishakhi Ray, Murali Krishna Ramanathan, Xiaofei Ma, and Anoop Deoras. 2024. CodeFort: Robust Training for Code Generation Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Stroudsburg, PA, USA, 5262–5277. doi:10.18653/v1/2024.findings-emnlp.303

[190] Zhuo Zhang, Guanhong Tao, Guangyu Shen, Shengwei An, Qiuling Xu, Yingqi Liu, Yapeng Ye, Yaoxuan Wu, and Xiangyu Zhang. 2023. PELICAN: Exploiting Backdoors of Naturally Trained Deep Learning Models in Binary Code Analysis. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Berkeley, CA, USA, 2365–2382. https://www.usenix.org/conference/usenixsecurity23/presentation/zhang-zhuo-pelican

[191] Ziyao Zhang, Yanlin Wang, Chong Wang, Jiachi Chen, and Zibin Zheng. 2024. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. arXiv preprint arXiv:2409.20550. arXiv:2409.20550 https://arxiv.org/html/2409.20550v1

[192] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. 2024. Models Are Codes: Towards Measuring Malicious Code Poisoning Attacks on Pre-trained Model Hubs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 2087–2098. doi:10.1145/3691620.3695271

[193] Shasha Zhou, Mingyu Huang, Yanan Sun, and Ke Li. 2024. Evolutionary Multi-Objective Optimization for Contextual Adversarial Example Generation. *Proceedings of the ACM on Software Engineering (FSE)* 1, FSE (2024), 2285–2308. doi:10.1145/3660808

[194] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 11 pages. https://proceedings.neurips.cc/paper_files/paper/2019/hash/49265d2447bc3bbfe9e76306ce40a31f-Abstract.html

[195] Yu Zhou, Xiaoqing Zhang, Juanjuan Shen, Tingting Han, Taolue Chen, and Harald Gall. 2022. Adversarial Robustness of Deep Code Comment Generation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4, Article 60 (2022), 30 pages. doi:10.1145/3501256

[196] Albert Ziegler. 2021. GitHub Copilot Research Recitation. GitHub Blog. https://github.blog/ai-and-ml/github-copilot/github-copilot-research-recitation/ Industry blog post, non-peer-reviewed.